

claude-windows / 985cc286-9e1e-4217-87c3-2d418bc4fa9c

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\985cc286-9e1e-4217-87c3-2d418bc4fa9c\subagents\agent-a31d0396f27efca03.jsonl`
- SHA-256: `7cbf21b469c977e0b541cb9d49e7e275ac396dc6b8459f2e6cf96908df41677d`
- Source modified: `2026-06-14T10:45:03+00:00`
- Imported at: `2026-07-05T16:48:25+00:00`
- project: `subagents`
- session_id: `985cc286-9e1e-4217-87c3-2d418bc4fa9c`

Transcript

1. user (2026-06-14T10:36:11.899Z)

Run the SERVED GATE-A VERIFICATION and add a repeatable served-regression check. Repo cwd C:/Users/avidu/Projects/badminton-highlight-indexer. ISOLATED worktree off origin/master: `git -C ... fetch origin` then `git -C ... worktree add C:/Users/avidu/Projects/bhi-servedverify -b feat/served-gate-a-regression origin/master`.

GOAL: confirm the Gate-A served-DEFAULT path (now `default\_segmenter=fusion\_hybrid` + `min\_crossings=2`, merged #146) holds on REAL footage ? not just the offline persisted-feature ablation. The flagged gap: the served `FusionSegmenter` path is mock-tested-only on real video. This is upstream of Gemini and (with the person cue OFF by default) needs NO GPU and NO Gemini ? it just re-scores cached WASB trajectories.

DATA LOCATION (important): the golden data lives in the MAIN checkout at `C:/Users/avidu/Projects/badminton-highlight-indexer/output/` ? your isolated worktree's `output/` is EMPTY (gitignored). Reference the main checkout's `output/human\_lovo\_manifest.json` (the 6 golden videos + their cached WASB trajectory CSV paths + golden label paths) and the cached trajectory CSVs by ABSOLUTE path. Do your CODE edits in your worktree.

STEP 1 ? understand the served path: read backend/pipeline/segmenters/fusion_hybrid.py (FusionSegmenter, _build_runner, _enrich_candidate_stats, _select_for_handoff), backend/pipeline/segmenters/base.py + trajectory_hybrid.py, backend/eval/fusion_golden.py, backend/eval/calibrate_wasb.py (load_golden_gts), backend/eval/metrics.py (F1 + segment scoring), and output/human_lovo_manifest.json. Determine how to run the PRODUCTION FusionSegmenter on a cached WASB trajectory CSV.

STEP 2 ? run it: for each of the 6 golden videos, run the served FusionSegmenter on its cached WASB trajectory with (A) the default config (min_crossings=2, person OFF = Gate-A ON) and (B) min_crossings=0 (Gate-A OFF, control). Score both vs golden labels (F1 + over-seg ratio) with the shipped metrics.

STEP 3 ? verdict: confirm the SERVED path reproduces the offline ablation ? Gate-A ON should give ~+0.074 ?F1, ~6/6 videos, over-seg ~1.68?1.01, within reasonable tolerance (the served path may differ slightly from the offline harness ? report any divergence HONESTLY). Does it HOLD (Gate-A improves F1 + cuts over-seg, no regression)? If YES ? the flip is verified on real footage. If a REGRESSION or material divergence ? flag it clearly with the numbers + likely cause (the flip is reversible).

STEP 4 ? make it REPEATABLE: add tests/test_served_gate_a_regression.py that SKIPS cleanly when the golden trajectories are absent (like the ablation litmus ? skips in CI, runs on the owner box) and asserts the served fusion-default path reproduces Gate-A's win within tolerance. Keep a small runner helper.

VERIFY: ruff + mypy + pytest green. PR: `gh pr create --base master`; MERGE on green (owner approved this verification). Branch-safe (explicit paths; `git log origin/master..HEAD` = only your commit). `Co-Authored-By: Claude Opus 4.8 (1M context) <noreply@anthropic.com>`. If you find a REAL regression, do NOT merge a misleading 'pass' ? report it + leave the PR open for

review.

Return: the served-path numbers table (Gate-A on vs off, per-video + aggregate), the VERDICT (HOLDS / regression / divergence), whether it matched the offline +0.074, the test added, and merge status.

2. assistant (2026-06-14T10:36:14.460Z)

I'll start by ramping up on the task. Let me first pull the latest in the main checkout and set up the isolated worktree, while reading the key files.

3. user (2026-06-14T10:36:17.897Z)

```
---HEAD---
e75ec35 feat(eval): LOS0/LOG0 ablation runner ? per-signal marginal ?F1 (Phase 0) (#147)
570cf2b feat(fusion): flip Gate-A (held-shuttle guard) to the served default ? config-reversible (#146)
7c6498f docs(quality): personalization Phase-0 checkpoint ? eval/safety rails + locked structural-guard decision (#145)
```

4. user (2026-06-14T10:36:17.940Z)

```
1      [
2      {
3          "name": "adarsh_avi_singles",
4          "trajectory": "C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/adarsh_avi_full_traj.csv",
5          "fps": 59.94, "frame_width": 1920,
6          "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden Labelled/Adarsh
and Avi.rallies.csv"
7      },
8      {
9          "name": "kushagra_singles",
10         "trajectory": "C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/kushagra_traj.csv",
11         "fps": 59.94, "frame_width": 1920,
12         "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden
Labelled/KushagraYashAviFirstVideo.rallies.csv"
13     },
14     {
15         "name": "targettest_doubles",
16         "trajectory": "C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/targettestvideo_traj.csv",
17         "fps": 29.97, "frame_width": 1920,
18         "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden
Labelled/LargeTestVideo.rallies.csv"
19     },
20     {
21         "name": "gbaaddy",
22         "trajectory": "C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/gbaaddy_traj.csv",
23         "fps": 59.94, "frame_width": 1920,
24         "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden
Labelled/gBaaddy20May2026.rallies.csv"
25     },
26     {
27         "name": "testlarge_short",
28         "trajectory": "C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/testlarge_traj.csv",
29         "fps": 29.97, "frame_width": 1920,
30         "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden
Labelled/TestLargeVideo.rallies.csv"
31     },
32     {
```

```

33         "name": "mahadevpura_singles",
34         "trajectory": "C:/Users/avidu/Projects/Annotation
Setup/Collect/Trajectories/mahadevpura_traj.csv",
35         "fps": 29.97, "frame_width": 1920,
36         "labels": "C:/Users/avidu/Projects/Annotation Setup/Collect/Golden
Labelled/MahadevpuraSingles.rallies.csv"
37     }
38 ]
39

```

5. assistant (2026-06-14T10:36:21.729Z)

The fetch worked. Now let me set up the isolated worktree and read the key files in parallel.

6. user (2026-06-14T10:36:23.614Z)

Preparing worktree (new branch 'feat/served-gate-a-regression')
branch 'feat/served-gate-a-regression' set up to track 'origin/master'.
HEAD is now at e75ec35 feat(eval): LOS0/LOG0 ablation runner ? per-signal marginal ?F1 (Phase 0)
(#147)

7. user (2026-06-14T10:36:23.793Z)

```

1     """Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).
2
3     Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
4     TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
5     do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
6     shuttle-seeded window via two overridable seams the engine exposes:
7
8     - ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
9     GPU-free, computed from the trajectory we already have); and, only when the person cue
10    is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
11    (`fusion_features`), persisting them into ``candidate_segments.stats`` for the learned
12    fusion (P3). The person path lazily builds ONE `PersonDetector` and only then loads
the
13    torchvision/supervision model ? so the default (person-OFF) path never loads a
detector
14    model nor touches the GPU. (torch/cv2 themselves are imported by the registry
regardless,
15    via the pre-existing yolo_hybrid ? person_detector chain; the lazy part is the model.)
16    - ``_select_for_handoff`` ? optional served Gate A/B: when
``indexing.fusion.min_crossings``
17    (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows
are
18    dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
19    **Persisted candidates remain the full superset** regardless, so the offline
experiment
20    harness can re-score any variant.
21
22    Defaults reproduce the substrate exactly: net_crossings/person features are persisted
but
23    nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config
seeds
24    and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the
default
25    segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift
through the
26    experiment harness (EXPERIMENT_HARNESS.md), never silently.
27    """
28    from typing import Any, Dict, List, Optional, Tuple
29
30    from backend.pipeline.segmenters.base import segmenter_registry
31    from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter

```

```

32     from backend.pipeline.detectors.base import DetectorRunner
33     from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
34     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
35     from backend.pipeline.detectors import rally_gate
36     from backend.pipeline.detectors import fusion_features as ff
37
38
39     class FusionSegmenter(TrajectoryHybridSegmenter):
40         """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate
41 B)."""
42
43         family = "fusion"
44         display_label = "Fusion"
45
46         def __init__(self, db: Any, config: Any):
47             super().__init__(db, config)
48             # Built lazily only if the person cue is enabled (no detector model loaded
49 otherwise).
50             self._person: Optional[Any] = None
51             # Cache the play-axis/net estimate per trajectory so rally_gate doesn't re-
52 estimate it
53             # over the whole trajectory once per candidate window (it depends only on
54 `points`).
55             self._axis_cache_key: Optional[int] = None
56             self._axis_cache: Optional[tuple] = None
57
58         @property
59         def name(self) -> str:
60             return "fusion_hybrid"
61
62         @property
63         def description(self) -> str:
64             return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate
65 rallies, "
66                 "net-crossing Gate A + optional person-occupancy Gate B adjudicate them,
67 and the "
68                 "AI provider sets semantic boundaries.")
69
70         def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
71 Any]]:
72             substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
73             if substrate == "tracknet":
74                 cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
75                 return TrackNetRunner(cfg), {
76                     "weights_path": cfg.weights_path,
77                     "queue_length": cfg.queue_length,
78                     "fusion_substrate": "tracknet",
79                 }
80             wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
81             return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate":
82 "wasb"}
83
84         def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
85 frame_width: float, video_path: str,
86 idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
87             stats = super()._enrich_candidate_stats(cw, points, fps, frame_width,
88 video_path, idx_cfg)
89             # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always
90 persisted
91             # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-
92 estimated
93             # over the whole trajectory by rally_gate (camera-agnostic); reused across
94 windows.
95             gated = rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps,
96 min_crossings=0,

```

```

estimate=self._axis_estimate(points))
85         stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0
86
87         fcfg = idx_cfg.get("fusion", {})
88         if fcfg.get("cue_flags", {}).get("person", False):
89             stats.update(self._person_features(cw, points, fps, frame_width, video_path,
fcfg))
90         return stats
91
92     def _axis_estimate(self, points: List[Any]) -> tuple:
93         """Cache rally_gate's (axis, net, span) play-axis estimate per trajectory so
it's
94         computed once per video, not once per candidate window (it depends only on
points)."""
95         key = id(points)
96         if self._axis_cache_key != key or self._axis_cache is None:
97             self._axis_cache_key = key
98             self._axis_cache = rally_gate.estimate_play_axis(points)
99         est = self._axis_cache
100         assert est is not None
101         return est
102
103     def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
104                         frame_width: float, video_path: str,
105                         fcfg: Dict[str, Any]) -> Dict[str, float]:
106         """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
107         frame range and compute the scale/translation-invariant person features. Lazily
builds
108         ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
109         # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
110         from backend.pipeline.detectors.person_detector import PersonDetector
111         if self._person is None:
112             self._person = PersonDetector(self.config)
113
114         frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
115         # round (not truncate): int() would map e.g. 1.999s*30 to frame 59 not 60 ? a
116         # window-edge off-by-one that drops the boundary frame from the shuttle
alignment.
117         start_f, end_f = round(cw["start"] * fps), round(cw["end"] * fps)
118         det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
119         fw = det["frame_width"] or frame_width
120         fh = det["frame_height"]
121         # If we can't trust the frame dims, every normalised feature would silently
collapse
122         # to ~0 ? return an explicit all-zero person-feature row instead of a half-baked
one.
123         if fw <= 0 or fh <= 0:
124             return {k: 0.0 for k in ff.PERSON_FEATURE_NAMES}
125
126         poly = fcfg.get("court_polygon")
127         polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
128         net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
129         shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
130         return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
131                                         court_polygon=polygon, net=net)
132
133     def _select_for_handoff(self, windows: List[Dict[str, Any]],
134                           window_stats: List[Dict[str, Any]],
135                           idx_cfg: Dict[str, Any]) -> List[int]:
136         """Served Gate A/B over the (already-enriched) per-window stats. Default
thresholds
137         (0) keep every window ? identical to the substrate. Persisted candidates are

```

```

138         untouched (full superset)."""
139         fcfg = idx_cfg.get("fusion", {})
140         min_crossings = int(fcfg.get("min_crossings", 0))
141         min_players = int(fcfg.get("min_players", 0))
142         person_on = bool(fcfg.get("cue_flags", {}).get("person", False))
143
144         keep: List[int] = []
145         for i, st in enumerate(window_stats):
146             if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
147                 continue
148             if person_on and min_players > 0 and st.get("players_in_court", 0.0) <
min_players:
149                 continue
150                 keep.append(i)
151         return keep
152
153
154     segmenter_registry.register(FusionSegmenter)
155

```

8. assistant (2026-06-14T10:36:26.792Z)

Worktree created. Now let me read the supporting files to understand the served path and scoring.

9. user (2026-06-14T10:36:27.724Z)

```

1     """Shared engine for trajectory-substrate hybrid segmenters.
2
3     tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
4     standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
5     now the single home for the whole pipeline shape:
6
7         healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
8         -> P3 AI-provider handoff over padded clips -> P4 DB population/linking
9
10    A concrete subclass supplies only its identity and runner wiring:
11    - ``family`` ? config section under ``indexing.*``, output subdir, and
12      the ``<family>-hybrid-<model>`` detector tag in metadata.
13    - ``display_label`` ? human label for log lines ("WASB", "TrackNet").
14    - ``name`` / ``description`` properties (the registry contract).
15    - ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
16      detection_params extras such as weights_path).
17
18    This base is deliberately NOT registered; only concrete subclasses register.
19    """
20    import os
21    import time
22    import logging
23    import concurrent.futures
24    from typing import Any, Dict, List, Optional, Tuple
25
26    import cv2
27
28    from backend.pipeline.segmenters.base import BasePlaySegmenter
29    from backend.utils.video import get_video_duration, extract_video_chunk
30    from backend.sports import sport_registry
31    from backend.providers import provider_registry
32    from backend.pipeline.detectors.base import DetectorRunner
33
34    logger = logging.getLogger(__name__)
35
36
37    def _fmt_ts(seconds: float) -> str:
38        seconds = max(0, int(seconds))

```

```

39     h, rem = divmod(seconds, 3600)
40     m, s = divmod(rem, 60)
41     return f"{h:02d}:{m:02d}:{s:02d}"
42
43
44     class TrajectoryHybridSegmenter(BasePlaySegmenter):
45         """Trajectory-detector hybrid: precise candidate rallies + AI semantic
46         boundaries."""
47
48         #: config section under `indexing` (velocity_threshold lives there), the
49         #: output subdir for trajectories, and the metadata detector-tag prefix.
50         family: str = ""
51         #: human-readable label used in log lines.
52         display_label: str = ""
53
54         def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str,
55 Any]]:
56
57             """Return (runner, detector-specific detection_params extras)."""
58             raise NotImplementedError
59
60         @staticmethod
61         def _probe_fps_width(video_path: str) -> tuple:
62             """Return (fps, frame_width) for a video, with sensible fallbacks."""
63             cap = cv2.VideoCapture(video_path)
64             fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
65             width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
66             cap.release()
67             return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)
68
69         @staticmethod
70         def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
71             """Provider-reported exchange count, else a duration-based estimate.
72
73             One canonical implementation ? the twins had drifted (wasb relied on
74             ``int(None)`` raising into the fallback; same result, by accident).
75             """
76             shot_count = segment.get("shuttle_exchange_count")
77             if shot_count is None:
78                 return max(3, int(duration / 0.8))
79             try:
80                 return int(shot_count)
81             except (ValueError, TypeError):
82                 return max(3, int(duration / 0.8))
83
84         # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged)
85         -----
86         def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
87             frame_width: float, video_path: str,
88             idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
89             """Stats dict persisted into ``candidate_segments.stats`` for one window. The
90             BASE
91             returns exactly the 4 motion keys, so the trajectory twins persist byte-
92             identical
93             rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
94             features). ``points`` are the whole-video trajectory points
95             (TrajectoryPoint)."""
96             return {
97                 "peak_velocity": cw["peak_velocity"],
98                 "mean_velocity": cw["mean_velocity"],
99                 "active_frame_count": cw["active_frame_count"],
100                 "track_id_count": cw["track_id_count"],
101             }
102
103         def _select_for_handoff(self, windows: List[Dict[str, Any]],
104             window_stats: List[Dict[str, Any]],

```

```

98             idx_cfg: Dict[str, Any]) -> List[int]:
99         """Indices of the candidate windows that proceed to the AI handoff (and thus may
100         become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
101         ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
102         Persisted candidates are NOT affected ? they remain the full superset for
offline
103         re-scoring."""
104         return list(range(len(windows)))
105
106     def process_video(self, video_id: str, video_path: str, # type: ignore[override]
107                     player_pool: Optional[List[str]] = None,
108                     max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
109         # override-ignore: the ABC declares the Result contract (Success|Failure)
110         # but every live segmenter still returns a bare list ? the documented
111         # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
112         label = self.display_label
113         # --- Sport profile + AI provider wiring (identical across segmenters) ---
114         sport_name = self.config.get("sport", "badminton")
115         try:
116             sport_profile = sport_registry.get(sport_name)
117         except KeyError as e:
118             logger.error(str(e))
119             return []
120
121         idx_cfg = self.config.get("indexing", {})
122         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
123         model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
124
125         api_key_env_var = f"{provider_name.upper()}_API_KEY"
126         api_key = os.environ.get(api_key_env_var)
127         if not api_key:
128             logger.error(
129                 f"{api_key_env_var} environment variable is not set! "
130                 f"To run the {provider_name} handoff, set your API key. "
131                 f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\"")
132             )
133             return []
134         try:
135             provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
136         except KeyError as e:
137             logger.error(str(e))
138             return []
139
140         video_duration = get_video_duration(video_path)
141         if video_duration <= 0:
142             logger.error("Invalid video duration.")
143             return []
144         fps, frame_width = self._probe_fps_width(video_path)
145
146         # --- Runner config + windowing thresholds ---
147         runner, runner_params = self._build_runner(idx_cfg)
148
149         velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
150         merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
151         min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
152         handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
153         ai_max_workers = idx_cfg.get("ai_max_workers", 5)
154         log_candidates = idx_cfg.get("log_yolo_candidates", True)
155         store_candidates = idx_cfg.get("store_yolo_candidates", True)
156
157         detection_params = {
158             "detector": self.name,

```



```

159         "velocity_thresh": velocity_thresh,
160         "merge_gap": merge_gap,
161         "min_action_window_duration": min_window_duration,
162         **runner_params,
163     }
164
165     # --- Phase 1: env healthcheck (fail fast with a clear message) ---
166     ok, msg = runner.healthcheck()
167     if not ok:
168         logger.error("%s WSL environment not ready: %s", label, msg)
169         return []
170     logger.info("%s WSL env OK: %s", label, msg)
171
172     # --- Phase 2: trajectory -> candidate rally windows ---
173     # Trajectory detectors process the whole video in one pass (they carry
174     # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
175     t_start = time.time()
176     out_dir = os.path.join("output", self.family)
177     csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
178     if not csv_path:
179         logger.error("%s produced no trajectory; aborting.", label)
180         return []
181
182     points = runner.parse_trajectory_csv(csv_path)
183     logger.info("Parsed %d trajectory points (%d visible).",
184               len(points), sum(1 for p in points if p.visible))
185
186     all_candidate_windows = runner.trajectory_to_action_windows(
187         points, fps=fps, frame_width=frame_width, chunk_start=0.0,
188         velocity_thresh=velocity_thresh, merge_gap=merge_gap,
189         min_window_duration=min_window_duration, chunk_index=0,
190     )
191     logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
192               label, time.time() - t_start, len(all_candidate_windows))
193
194     if log_candidates:
195         for cw in all_candidate_windows:
196             logger.info(f"[{label} rally]
197 {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
198 f"({cw['end'] - cw['start']:.1f}s) |
199 frames={cw['active_frame_count']} "
200 f"peak_v={cw['peak_velocity']:.3f}
201 mean_v={cw['mean_velocity']:.3f}")
202
203     # Per-window stats via the enrichment hook ? base = the 4 motion keys; the
204     # segmenter adds net_crossings + person-cue features. Computed once, reused for
205     # persistence and the handoff gate below.
206     window_stats = [
207         self._enrich_candidate_stats(cw, points, fps, frame_width, video_path,
208         idx_cfg)
209         for cw in all_candidate_windows
210     ]
211
212     # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
213     candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
214     if store_candidates:
215         for i, cw in enumerate(all_candidate_windows):
216             candidate_ids[i] = self.db.add_candidate_segment(
217                 video_id, detector=self.name,
218                 start_time=cw["start"], end_time=cw["end"],
219                 chunk_index=cw["chunk_index"], confidence=min(1.0,
220                 cw["peak_velocity"]),
221                 stats=window_stats[i], detection_params=detection_params,

```

```

217         )
218
219     if not all_candidate_windows:
220         return []
221
222     # --- Phase 3: AI provider handoff over padded candidate clips ---
223     # Which windows reach the handoff (base = all; fusion may apply Gate A/B).
Persisted
224     # candidates above stay the full superset ? only the served play_segments are
gated.
225     handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats,
idx_cfg)
226     handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
227     os.makedirs(handoff_dir, exist_ok=True)
228     logger.info("Phase 3: %s validation on %d candidate windows...",
229                 provider_name, len(handoff_indices))
230
231     def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
232         w_start = max(0, window["start"] - handoff_padding)
233         w_end = min(video_duration, window["end"] + handoff_padding)
234         w_dur = w_end - w_start
235         w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
236         if not extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True):
237             return []
238         prompt = sport_profile.get_prompt(chunk_duration=w_dur)
239         segments, _ = provider.analyze_video(w_path, prompt, chunk_duration=w_dur,
240                                             label=f"Handoff {wi+1}")
241         valid = []
242         for seg in segments:
243             try:
244                 seg["start_time"] = float(seg["start_time"]) + w_start
245                 seg["end_time"] = float(seg["end_time"]) + w_start
246                 seg["_candidate_id"] = candidate_ids[wi]
247                 valid.append(seg)
248             except (ValueError, TypeError, KeyError) as e:
249                 logger.warning("Dropping segment with unparseable timestamps: %s ?
%s", seg, e)
250             try:
251                 os.remove(w_path)
252             except OSError:
253                 pass
254         return valid
255
256     ai_segments: List[dict] = []
257     with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
258         futures = [executor.submit(process_candidate_window, wi,
all_candidate_windows[wi])
259                     for wi in handoff_indices]
260         for future in futures:
261             ai_segments.extend(future.result())
262         try:
263             os.rmdir(handoff_dir)
264         except OSError:
265             pass
266         logger.info("Phase 3 completed. AI confirmed %d semantic play segments.",
len(ai_segments))
267
268     # --- Phase 4: DB population + candidate linking ---
269     segments_data = []
270     for segment in ai_segments:
271         try:
272             start = float(segment["start_time"])
273             end = float(segment["end_time"])

```

```

274         dur = end - start
275         metadata = {
276             "shot_count": self._shot_count_from(segment, dur),
277             "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
278             "detector": f"{self.family}-hybrid-{model_name}",
279         }
280         segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
281         if segment_id:
282             candidate_id = segment.get("_candidate_id")
283             if candidate_id is not None:
284                 self.db.link_candidate_to_segment(candidate_id, segment_id)
285             segments_data.append({
286                 "id": segment_id, "start_time": start, "end_time": end,
287                 "duration": dur, "metadata": metadata,
288             })
289         except Exception as e:
290             logger.warning("Skipping invalid segment struct: %s", e)
291
292     return segments_data
293

```

10. user (2026-06-14T10:36:27.941Z)

```

1     """Offline golden-set person-feature extractor ? the fusion-P3 measurement's GPU step.
2
3     Mirrors `distill_local`'s CSV-driven flow but ADDS the ~5 person-cue features per
shuttle
4     candidate window, by running `PersonDetector` over the ORIGINAL video. It reuses the
5     EXISTING golden trajectory CSVs (so **no WASB re-run**), needs **no Gemini and no DB**,
and
6     writes one small JSON per video. The LOVO comparison (`fusion_compare.py`) then runs
100% on
7     CPU offline. So the GPU touches exactly one thing ? person detection ? and the lift is
8     measured by the shipped experiment harness; person features enter only as feature
columns the
9     learned scorer weights (no special-casing, default-OFF until the golden ?F1 justifies
them).
10
11     Run (on the GPU box):
12         python -m backend.eval.fusion_golden --manifest output/human_lovo_manifest.json
--out-dir output
13
14     Each output `output/<name>_golden_features.json` is the replayable feature substrate ?
re-run
15     `fusion_compare` with different variants/thresholds forever without re-detecting on the
GPU.
16     """
17     from __future__ import annotations
18
19     import argparse
20     import json
21     import os
22     import time
23     from typing import Any, Dict, List, Optional
24
25     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
26     from backend.pipeline.detectors import rally_gate
27     from backend.pipeline.detectors import fusion_features as ff
28     from backend.eval import distill_local
29     from backend.eval.calibrate_wasb import load_golden_gts
30     from backend.eval.training import label_candidates
31     from backend.utils.run_telemetry import RunTelemetry
32

```

```

33     # The 5 fps/resolution-invariant shuttle features distill_local already computes.
34     SHUTTLE_FEATURE_NAMES = list(distill_local.FEATURE_NAMES)
35     # The 5 person-cue features (fusion_features.PERSON_FEATURE_NAMES).
36     PERSON_FEATURE_NAMES = list(ff.PERSON_FEATURE_NAMES)
37
38
39     def _derive_video_path(spec: Dict[str, Any]) -> str:
40         """Original MP4 for a manifest entry ? explicit `video_path`, else derived from the
41         golden labels path (`X.rallies.csv` -> `X.MP4`, the golden-set naming
42         convention)."""
43         if spec.get("video_path"):
44             return str(spec["video_path"])
45         labels = str(spec["labels"])
46         if labels.endswith(".rallies.csv"):
47             return labels[: -len(".rallies.csv")] + ".MP4"
48         raise ValueError(f"{spec.get('name')}: no video_path and labels {labels!r} isn't
49         *.rallies.csv")
50
51     def extract_video(spec: Dict[str, Any], detector: Optional[Any] = None,
52                      frame_skip: int = 3, iou: float = 0.5,
53                      log_every_sec: float = 15.0,
54                      telemetry: Optional[Any] = None) -> Dict[str, Any]:
55         """One golden video -> a feature record (shuttle + person features per candidate
56         window).
57         Returns `{name, fps, frame_width,
58         candidates:[{start,end,shuttle{,person{,label}], gts}``.
59         `detector` (anything with `detections_over_windows`) is injectable for GPU-free
60         tests;
61         when None a real `PersonDetector` is built (loads torchvision on first use).
62         `telemetry` (a `RunTelemetry` or None) records per-frame velocity + machine-health
63         snapshots
64         from the detection progress callback so a hard kill leaves a forensic runlog.
65         """
66         fps, fw = float(spec["fps"]), float(spec["frame_width"])
67         traj = str(spec["trajectory"])
68         points = TrackNetRunner.parse_trajectory_csv(traj)
69         intervals, x_shuttle = distill_local.build_candidates(traj, fps, fw)
70         gts = load_golden_gts(str(spec["labels"]))
71         labels = label_candidates(intervals, gts, iou)
72         # Net axis for the person two-sided split ? reuse rally_gate's camera-agnostic
73         estimate
74         # (parity with the shuttle net-crossing gate) rather than hard-coding an
75         orientation.
76         axis, net_px, _span = rally_gate.estimate_play_axis(points)
77         video_path = _derive_video_path(spec)
78
79         if detector is None:
80             from backend.pipeline.detectors.person_detector import PersonDetector
81             detector = PersonDetector({"indexing": {"use_gpu": True}})
82
83         # Single-pass person detection over ALL candidate windows (open the video ONCE, read
84         # forward ? no per-window seek). Then compute features per window (pure, no GPU).
85         n = len(intervals)
86         win_frames = [(round(s * fps), round(e * fps)) for (s, e) in intervals]
87         print(f"[fusion_golden] {spec['name']}: {n} windows ? person-detecting in one
88         pass...", flush=True)
89         t0 = time.monotonic()
90         state = {"last": t0}
91
92         def _progress(done: int, total: int) -> None:
93             if telemetry is not None:
94                 telemetry.snapshot(done, total, phase=spec["name"])
95             now = time.monotonic()

```

```

89         if now - state["last"] >= log_every_sec or done >= total:
90             el = now - t0
91             rate = done / el if el > 0 else 0.0
92             eta = (total - done) / rate if rate > 0 else 0.0
93             pct = 100 * done // total if total else 0
94             print(f"[fusion_golden]    {spec['name']}: detect frame {done}/{total}
({pct}%) "
95                     f"| {el:.0f}s elapsed | ETA {eta:.0f}s", flush=True)
96             state["last"] = now
97
98     per_window = detector.detections_over_windows(video_path, win_frames, frame_skip,
99                                                    progress_cb=_progress)
100
101     candidates: List[Dict[str, Any]] = []
102     for i, ((s, e), xs) in enumerate(zip(intervals, x_shuttle)):
103         det = per_window[i]
104         det_fw = det.get("frame_width") or fw
105         det_fh = det.get("frame_height") or 0.0
106         # Normalise the (pixel) net coord to 0-1 on the play axis, matching the foot-
point
107         # normalisation fusion_features uses (x/width, y/height).
108         if axis == "x":
109             net_norm = net_px / det_fw if det_fw > 0 else 0.5
110         else:
111             net_norm = net_px / det_fh if det_fh > 0 else 0.5
112         sf, ef = win_frames[i]
113         shuttle_in = [p for p in points if sf <= p.frame <= ef]
114         person = ff.compute_person_features(det["frames"], shuttle_in, det_fw, det_fh,
115                                             court_polygon=None, net=(axis, net_norm))
116         candidates.append({
117             "start": float(s), "end": float(e),
118             "shuttle": {k: float(v) for k, v in zip(SHUTTLE_FEATURE_NAMES, xs)},
119             "person": {k: float(v) for k, v in person.items()},
120             "label": int(labels[i]),
121         })
122     return {
123         "name": spec["name"], "fps": fps, "frame_width": fw,
124         "candidates": candidates,
125         "gts": [[float(a), float(b)] for a, b in gts],
126     }
127
128
129     def run(manifest_path: str, out_dir: str, frame_skip: int = 3, iou: float = 0.5,
130            fresh: bool = False, smallest_first: bool = False) -> List[str]:
131         with open(manifest_path, encoding="utf-8") as f:
132             specs = json.load(f)
133         if smallest_first:
134             # Process small videos first for quick feedback (the slow full match lands
last);
135             # the LOVO result is order-independent. Proxy size by the trajectory CSV (one
row/frame).
136             specs = sorted(specs, key=lambda s: (os.path.getsize(s["trajectory"])
137                                                  if os.path.isfile(s.get("trajectory", ""))
else 0))
138         os.makedirs(out_dir, exist_ok=True)
139         # Run-telemetry black box: a long GPU run can be OS-killed with no traceback, so
keep a tiny
140         # rotating runlog of velocity + machine health for the post-mortem. Constructing it
must never
141         # break extraction, so guard the build and fall back to no telemetry on any error.
142         telem: Optional[RunTelemetry]
143         try:
144             telem = RunTelemetry(os.path.join(out_dir, "run_telemetry.jsonl"),
label="fusion_golden")
145         except Exception: # noqa: BLE001 - telemetry wiring must never break the run it

```

```

observes
146         telem = None
147     written: List[str] = []
148     for vi, spec in enumerate(specs, 1):
149         out = os.path.join(out_dir, f"{spec['name']}_golden_features.json")
150         # Resumable: skip a video whose JSON already exists (a re-run picks up where a
prior
151         # run left off instead of re-detecting). --fresh forces a redo.
152         if not fresh and os.path.isfile(out):
153             print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: already
extracted "
154                 f"-> {out} (use --fresh to redo)", flush=True)
155             written.append(out)
156             continue
157         print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: extracting...",
flush=True)
158         t0 = time.monotonic()
159         if telem is not None:
160             telem.event("video_start", spec["name"])
161         try:
162             data = extract_video(spec, frame_skip=frame_skip, iou=iou, telemetry=telem)
163         except Exception as e: # noqa: BLE001 - record machine state at death, then re-
raise
164             if telem is not None:
165                 telem.event("error", spec["name"], err=str(e))
166             raise
167         n_pos = sum(c["label"] for c in data["candidates"])
168         with open(out, "w", encoding="utf-8") as f:
169             json.dump(data, f, indent=2)
170         if telem is not None:
171             telem.event("video_done", spec["name"],
172                         windows=len(data["candidates"]), positives=n_pos)
173         print(f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: DONE in
{time.monotonic() - t0:.0f}s "
174             f"| {len(data['candidates'])} windows ({n_pos} positive) |
{len(data['gts'])} golden rallies "
175             f"-> {out}", flush=True)
176         written.append(out)
177     return written
178
179
180     def main() -> None:
181         ap = argparse.ArgumentParser(description="Extract golden-set shuttle+person features
(fusion P3 GPU step).")
182         ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
183         ap.add_argument("--out-dir", default="output")
184         ap.add_argument("--frame-skip", type=int, default=3)
185         ap.add_argument("--iou", type=float, default=0.5)
186         ap.add_argument("--fresh", action="store_true", help="re-extract even if a JSON
already exists")
187         ap.add_argument("--smallest-first", action="store_true",
188                         help="process smaller videos first for quick feedback (LOVO is
order-independent)")
189         args = ap.parse_args()
190         run(args.manifest, args.out_dir, args.frame_skip, args.iou, fresh=args.fresh,
191             smallest_first=args.smallest_first)
192
193
194     if __name__ == "__main__":
195         main()
196

```

11. assistant (2026-06-14T10:36:32.004Z)

Now let me read the key dependencies: `calibrate_wasb` (load_golden_gts + how candidates are

built), metrics, distill_local (build_candidates), and rally_gate.

12. user (2026-06-14T10:36:32.578Z)

```
1      """Per-video calibration driver: WASB trajectory + golden rallies -> tuned thresholds +
2      "+X%".
3
4      Wires the already-tested pieces together:
5      WASB trajectory CSV (Frame,Visibility,X,Y)
6      -> TrackNetRunner.trajectory_to_action_windows(cfg)    [predict_fn]
7      -> backend.eval.calibration (improvement_report / cross_validate)
8
9      Run (after a full-video WASB pass exists):
10     python -m backend.eval.calibrate_wasb --trajectory full_traj.csv \
11         --labels "golden_labels_badminton.csv" --fps 59.94 --frame-width 1920
12
13     Honesty note: the grid is *selected* on the full GT, so `improvement_report` on that
14     same GT is an **optimistic** upper bound. The trustworthy number is the
15     cross-validated held-out **recall** (and, once a 2nd labelled video exists,
16     `leave_one_video_out` for precision/F1 generalization).
17     """
18
19     from __future__ import annotations
20
21     import argparse
22     from typing import List, Tuple, Callable
23
24     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
25     from backend.eval.calibration import (
26         select_best, improvement_report, cross_validate, evaluate,
27     )
28
29     Interval = Tuple[float, float]
30     Config = Tuple[float, float, float]    # (velocity_thresh, merge_gap,
31     min_window_duration)
32
33     BASELINE: Config = (0.02, 2.0, 2.0)    # the WasbConfig / trajectory_to_action_windows
34     defaults
35
36     def load_golden_gts(csv_path: str) -> List[Interval]:
37         """Rally [start,end] seconds from a golden CSV ? delegates to the canonical loader.
38
39         Accepts BOTH golden conventions now (start_time/end_time AND the collector export's
40         start,end ? the historical fork is closed; see gt_loader.py / ADR-008). Keeps this
41         driver's legacy lenient semantics (skip bad rows), but skipped rows are now logged.
42         """
43         from backend.eval.gt_loader import load_gt_intervals
44         return load_gt_intervals(csv_path, strict=False)
45
46     def make_predict_fn(trajectory_csv: str, fps: float, frame_width: float) ->
47     Callable[[Config], List[Interval]]:
48         """predict_fn(cfg) -> rally windows, from a cached WASB trajectory (parsed once)."""
49         points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
50
51         def predict_fn(cfg: Config) -> List[Interval]:
52             vt, mg, mwd = cfg
53             wins = TrackNetRunner.trajectory_to_action_windows(
54                 points, fps=fps, frame_width=frame_width,
55                 velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd,
56             )
57             return [(w["start"], w["end"]) for w in wins]
58
59     return predict_fn
```

```

58
59 def default_grid() -> List[Config]:
60     return [(vt, mg, mwd)
61             for vt in (0.005, 0.01, 0.02, 0.03, 0.05)
62             for mg in (1.0, 2.0, 3.0)
63             for mwd in (1.0, 2.0, 3.0)]
64
65
66 def run(trajjectory_csv: str, labels_csv: str, fps: float, frame_width: float,
67        iou: float = 0.5) -> dict:
68     gts = load_golden_gts(labels_csv)
69     if not gts:
70         raise SystemExit(f"no usable golden rallies in {labels_csv}")
71     predict_fn = make_predict_fn(trajjectory_csv, fps, frame_width)
72     grid = default_grid()
73
74     base = evaluate(predict_fn(BASELINE), gts, iou)
75     best_cfg, best_f1 = select_best(predict_fn, grid, gts, metric="f1",
iou_threshold=iou)
76     fit = improvement_report(predict_fn, BASELINE, best_cfg, gts, iou) # OPTIMISTIC
(fit on full GT)
77     cv = cross_validate(predict_fn, grid, gts, k=5) # HONEST held-
out recall
78
79     print("=== WASB rally-segmentation calibration ===")
80     print(f"golden rallies: {len(gts)} | IoU thresh: {iou}")
81     print(f"baseline cfg {BASELINE}: precision={base.precision:.3f}
recall={base.recall:.3f} "
82           f"f1={base.f1:.3f} meanIoU={base.mean_iou:.3f}")
83     print(f"tuned cfg {best_cfg}: f1={best_f1:.3f}")
84     print(f" precision {fit['baseline']['precision']:.3f} ->
{fit['tuned']['precision']:.3f} "
85           f"({fit['precision_delta_pct']:+.1f}% , fit-on-full = optimistic)")
86     print(f" f1 {fit['baseline']['f1']:.3f} -> {fit['tuned']['f1']:.3f} "
87           f"({fit['f1_delta_pct']:+.1f}%)")
88     print(f"HONEST (cross-val, {cv.get('folds','?')} folds): held-out recall "
89           f"{cv.get('mean_test',float('nan')):.3f} ± {cv.get('std_test',0):.3f}, "
90           f"generalization_gap {cv.get('generalization_gap',float('nan')):+.3f}")
91     print("(For precision/F1 generalization, add a 2nd labelled video ->
leave_one_video_out.)")
92     return {"baseline": base.as_dict(), "best_cfg": best_cfg, "fit": fit, "cv": cv}
93
94
95 def main():
96     ap = argparse.ArgumentParser()
97     ap.add_argument("--trajectory", required=True, help="WASB trajectory CSV
(Frame,Visibility,X,Y)")
98     ap.add_argument("--labels", required=True, help="golden labels CSV
(start_time/end_time cols)")
99     ap.add_argument("--fps", type=float, default=59.94)
100    ap.add_argument("--frame-width", type=float, default=1920.0)
101    ap.add_argument("--iou", type=float, default=0.5)
102    args = ap.parse_args()
103    run(args.trajectory, args.labels, args.fps, args.frame_width, args.iou)
104
105
106 if __name__ == "__main__":
107     main()
108

```

13. user (2026-06-14T10:36:32.915Z)

```

1     """Pure temporal interval-matching metrics for rally evaluation.
2
3     No I/O, no DB, no global state ? every function here is deterministic and unit

```



```

4      tested. An "interval" is a `(start, end)` tuple of floats in seconds with
5      start < end. We score a list of *predicted* rally intervals against a list of
6      *ground-truth* intervals via temporal IoU and one-to-one greedy matching.
7      """
8
9      from dataclasses import dataclass, field
10     from typing import List, Tuple, Optional
11
12     Interval = Tuple[float, float]
13
14
15     def interval_iou(a: Interval, b: Interval) -> float:
16         """Temporal intersection-over-union of two intervals.
17
18         Returns 0.0 when they don't overlap (or either is degenerate). IoU is
19         overlap / union, both measured as durations on the timeline.
20         """
21         a_start, a_end = a
22         b_start, b_end = b
23         if a_end <= a_start or b_end <= b_start:
24             return 0.0
25         inter = max(0.0, min(a_end, b_end) - max(a_start, b_start))
26         if inter <= 0.0:
27             return 0.0
28         union = (a_end - a_start) + (b_end - b_start) - inter
29         return inter / union if union > 0 else 0.0
30
31
32     @dataclass
33     class Match:
34         """One matched predicted?ground-truth pair."""
35         pred_index: int
36         gt_index: int
37         iou: float
38
39
40     @dataclass
41     class MatchResult:
42         """Outcome of matching predictions to ground truth at a given IoU threshold."""
43         matches: List[Match] = field(default_factory=list)
44         unmatched_pred_indices: List[int] = field(default_factory=list) # false positives
45         unmatched_gt_indices: List[int] = field(default_factory=list)   # false negatives
46     (misses)
47
48     def match_intervals(preds: List[Interval], gts: List[Interval],
49                       iou_threshold: float = 0.5) -> MatchResult:
50         """Greedy one-to-one matching of predictions to ground truth by descending IoU.
51
52         Every candidate (pred, gt) pair with IoU >= threshold is considered; pairs
53         are claimed highest-IoU first, and each prediction/GT can be used once. This
54         keeps a single prediction from being credited for two ground-truth rallies
55         (and vice versa).
56         """
57         candidates = []
58         for pi, p in enumerate(preds):
59             for gi, g in enumerate(gts):
60                 iou = interval_iou(p, g)
61                 # Require real overlap: at iou_threshold=0.0 a plain `iou >= threshold`
62                 # match completely disjoint intervals (IoU 0.0), fabricating perfect scores.
63                 if iou > 0.0 and iou >= iou_threshold:
64                     candidates.append((iou, pi, gi))
65                 # Highest IoU first; ties broken by indices for determinism.
66                 candidates.sort(key=lambda c: (-c[0], c[1], c[2]))

```

```

67
68     used_pred, used_gt = set(), set()
69     matches: List[Match] = []
70     for iou, pi, gi in candidates:
71         if pi in used_pred or gi in used_gt:
72             continue
73         used_pred.add(pi)
74         used_gt.add(gi)
75         matches.append(Match(pred_index=pi, gt_index=gi, iou=iou))
76
77     unmatched_pred = [i for i in range(len(preds)) if i not in used_pred]
78     unmatched_gt = [i for i in range(len(gts)) if i not in used_gt]
79     return MatchResult(matches=matches,
80                        unmatched_pred_indices=unmatched_pred,
81                        unmatched_gt_indices=unmatched_gt)
82
83
84 @dataclass
85 class Score:
86     """Aggregate metrics for one (predictions vs ground-truth) evaluation."""
87     iou_threshold: float
88     num_pred: int
89     num_gt: int
90     true_positives: int
91     false_positives: int
92     false_negatives: int
93     precision: float
94     recall: float
95     f1: float
96     mean_iou: float          # over matched pairs (0.0 if none)
97     mean_start_error: float  # mean |pred_start - gt_start| over matches, seconds
98     mean_end_error: float    # mean |pred_end - gt_end| over matches, seconds
99
100     def as_dict(self) -> dict:
101         return self.__dict__.copy()
102
103
104 def score(preds: List[Interval], gts: List[Interval],
105          iou_threshold: float = 0.5,
106          match_result: Optional[MatchResult] = None) -> Score:
107     """Compute precision/recall/F1, mean matched IoU, and boundary errors.
108
109     Pass a precomputed `match_result` to avoid re-matching (e.g. when also
110     extracting the failure lists); otherwise matching is done here.
111     """
112     mr = match_result if match_result is not None else match_intervals(preds, gts,
113 iou_threshold)
114
115     tp = len(mr.matches)
116     fp = len(mr.unmatched_pred_indices)
117     fn = len(mr.unmatched_gt_indices)
118
119     precision = tp / (tp + fp) if (tp + fp) > 0 else 0.0
120     recall = tp / (tp + fn) if (tp + fn) > 0 else 0.0
121     f1 = (2 * precision * recall / (precision + recall)) if (precision + recall) > 0
122 else 0.0
123
124     if mr.matches:
125         mean_iou = sum(m.iou for m in mr.matches) / tp
126         mean_start_err = sum(abs(preds[m.pred_index][0] - gts[m.gt_index][0]) for m in
127 mr.matches) / tp
128         mean_end_err = sum(abs(preds[m.pred_index][1] - gts[m.gt_index][1]) for m in
129 mr.matches) / tp
130     else:
131         mean_iou = mean_start_err = mean_end_err = 0.0

```

```

128
129         return Score(
130             iou_threshold=iou_threshold,
131             num_pred=len(preds), num_gt=len(gts),
132             true_positives=tp, false_positives=fp, false_negatives=fn,
133             precision=precision, recall=recall, f1=f1,
134             mean_iou=mean_iou, mean_start_error=mean_start_err, mean_end_error=mean_end_err,
135         )
136
137
138     def pr_curve(preds: List[Interval], gts: List[Interval],
139                 thresholds: Optional[List[float]] = None) -> List[Score]:
140         """Score across a sweep of IoU thresholds (default 0.1..0.9 step 0.1).
141
142         Lets you see strict-vs-lenient behaviour rather than a single operating point.
143         """
144         if thresholds is None:
145             thresholds = [round(0.1 * i, 1) for i in range(1, 10)]
146         return [score(preds, gts, iou_threshold=t) for t in thresholds]
147

```

14. assistant (2026-06-14T10:36:43.253Z)

Now let me look at the offline ablation harness (the +0.074 source), build_candidates, rally_gate, and the existing ablation/litmus test pattern. Let me also find the offline fusion comparison that produced the +0.074 number.

15. user (2026-06-14T10:36:45.469Z)

Found 38 files

- tests\test_ablation.py
- docs\ABLATION_LAYER.md
- backend\eval\ablation.py
- tests\test_fusion_hybrid.py
- tests\test_config.py
- eval_baselines\experiment_leaderboard.json
- docs\QUALITY_ITERATIONS.md
- config.json
- backend\config\models.py
- docs\PERSONALIZATION_PLAN.md
- tests\test_per_user_regression.py
- tests\test_per_user_eval.py
- backend\eval\per_user_eval.py
- backend\eval\fusion_golden.py
- docs\CODE_MAP.md
- tests\test_experiment_honest.py
- docs\archives\quality-iterations\README.md
- docs\archives\quality-iterations\2026-06-first-ab-experiment\README.md
- docs\EXPERIMENT_HARNESS.md
- backend\eval\experiment.py
- tests\test_fusion_compare.py
- backend\eval\fusion_compare.py
- backend\eval\fusion_audio.py
- backend\eval\segmentation_metrics.py
- tests\test_segmentation_metrics.py
- docs\ANALYZERS_AND_RECONCILERS.md
- tests\test_rally_gate.py
- docs\NEXT_STEPS.md
- docs\MULTI_SIGNAL_FUSION_PLAN.md
- backend\pipeline\detectors\rally_gate.py
- backend\pipeline\segmenters\fusion_hybrid.py
- tests\test_experiment.py
- backend\eval\calibrate_local.py
- backend\eval\distill_local.py
- backend\eval\eval_partial_labels.py

```
backend\eval\export_wasb_segments.py
tests\test_calibrate_local.py
tests\test_export_wasb_segments.py
```

16. user (2026-06-14T10:36:45.852Z)

```
1      """Rally-start gate: suppress between-point false fires using shuttle net-crossings.
2
3      WASB (shuttle-only) over-detects: when a player flicks/tosses the shuttle back to
4      the opponent for service after losing a point, that single trajectory looks like a
5      rally. The discriminator is structural, not duration-based: a real rally has the
6      shuttle crossing the net MULTIPLE times (each shot lands on the other side); a
7      single service feed crosses ~once.
8
9      This module is camera-agnostic and needs NO new model ? it works on the trajectory
10     we already produce (Frame,Visibility,X,Y). It estimates the play axis (the image
11     axis with the larger shuttle spread) and the net position (median shuttle coord on
12     that axis ? the shuttle clusters at the two ends and crosses the net region often,
13     so the median sits near the net), then counts sign changes of (coord - net) across
14     the visible points inside a candidate window, with a deadband to ignore jitter.
15
16     Gate A in the over-fire fix spectrum (B = player-stance state machine, future).
17     Pure functions only ? no I/O, unit-tested; intended as a post-filter on the
18     windows from trajectory_to_action_windows, or to fold into it later.
19     """
20     from __future__ import annotations
21
22     from dataclasses import dataclass
23     from typing import List, Optional, Sequence, Tuple
24
25     Interval = Tuple[float, float]
26
27
28     @dataclass
29     class GatedWindow:
30         start: float
31         end: float
32         crossings: int
33         visible_points: int
34         kept: bool
35
36
37     def _spread(vals: Sequence[float]) -> float:
38         """Robust spread = p95 - p5 (ignores a few outlier detections)."""
39         if len(vals) < 2:
40             return 0.0
41         s = sorted(vals)
42         lo = s[max(0, int(0.05 * (len(s) - 1)))]
43         hi = s[min(len(s) - 1, int(0.95 * (len(s) - 1)))]
44         return hi - lo
45
46
47     def _median(vals: Sequence[float]) -> float:
48         if not vals:
49             return 0.0
50         s = sorted(vals)
51         n = len(s)
52         return s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2])
53
54
55     def estimate_play_axis(points) -> Tuple[str, float, float]:
56         """Return (axis 'x'/'y', net_value, span) from visible points.
57
58         Play axis = the image axis along which the shuttle travels most (larger spread).
59         For a baseline camera that's Y (players top/bottom); for a side camera, X.
```

```

60     """
61     xs = [p.x for p in points if p.visible]
62     ys = [p.y for p in points if p.visible]
63     sx, sy = _spread(xs), _spread(ys)
64     if sx >= sy:
65         return "x", _median(xs), sx
66     return "y", _median(ys), sy
67
68
69 def count_net_crossings(window_points, axis: str, net: float, deadband: float) -> int:
70     """Count sign changes of (coord - net) across visible points, ignoring the
71     deadband zone near the net (so jitter at the net doesn't inflate the count)."""
72     crossings = 0
73     last_side = None
74     for p in window_points:
75         if not p.visible:
76             continue
77         v = (p.x if axis == "x" else p.y) - net
78         if abs(v) < deadband:
79             continue
80         side = v > 0
81         if last_side is not None and side != last_side:
82             crossings += 1
83         last_side = side
84     return crossings
85
86
87 def _points_in(points, fps: float, start: float, end: float):
88     return [p for p in points if start * fps <= p.frame <= end * fps]
89
90
91 def gate_windows(points, windows: List[Interval], fps: float,
92                 min_crossings: int = 2, deadband_frac: float = 0.06,
93                 estimate: Optional[Tuple[str, float, float]] = None) ->
List[GatedWindow]:
94     """Annotate each (start,end) window with its shuttle net-crossing count and a
95     keep decision (crossings >= min_crossings). Deadband is a fraction of the play
96     axis span. Net/axis are estimated once over the whole trajectory (stable).
97
98     `estimate` lets a caller scoring MANY windows off one trajectory pass a precomputed
99     `(axis, net, span)` (from `estimate_play_axis`) so the  $O(P \log P)$  estimate isn't
redone
100     per window ? same result, avoids the per-window re-estimation."""
101     axis, net, span = estimate if estimate is not None else estimate_play_axis(points)
102     deadband = deadband_frac * span
103     out: List[GatedWindow] = []
104     for (s, e) in windows:
105         wp = _points_in(points, fps, s, e)
106         c = count_net_crossings(wp, axis, net, deadband)
107         vis = sum(1 for p in wp if p.visible)
108         out.append(GatedWindow(start=s, end=e, crossings=c, visible_points=vis,
109                               kept=c >= min_crossings))
110     return out
111
112
113 def apply_gate(points, windows: List[Interval], fps: float,
114               min_crossings: int = 2, deadband_frac: float = 0.06) -> List[Interval]:
115     """Convenience: return only the windows that pass the crossing gate."""
116     return [(g.start, g.end) for g in
117             gate_windows(points, windows, fps, min_crossings, deadband_frac) if g.kept]
118

```

17. assistant (2026-06-14T10:36:53.101Z)

Now let me read the offline ablation/fusion_compare that produced the +0.074, and the existing

ablation litmus test (the skip pattern to mirror), and `distill_local.build_candidates`.

18. user (2026-06-14T10:36:54.061Z)

```
1      """Pure-CPU LOVO comparison: shuttle-only vs shuttle+person learned fusion on the golden
2      set.
3      The fusion-P3 measurement. Loads the per-video feature JSONs written by
4      `fusion_golden.py`
5      and runs the SHIPPED experiment harness (`experiment.compare`, honest leave-one-video-
6      out) ?
7      no new scoring/labeling math. The two variants differ ONLY by `feature_names` (the
8      harness's
9      designed seam): the person features are extra columns the learned LogisticRegression
10     weights,
11     so the held-out ?F1 is an honest answer to "does the person cue help?" ? not a hunch.
12
13     Run (no GPU):
14     python -m backend.eval.fusion_compare --features-dir output --record
15     """
16     from __future__ import annotations
17
18     import argparse
19     import glob
20     import json
21     import os
22     from typing import Any, Dict, List
23
24     from backend.eval import experiment
25     from backend.eval.eval_stats import paired_delta_ci
26     from backend.eval.fusion_golden import PERSON_FEATURE_NAMES, SHUTTLE_FEATURE_NAMES
27
28     def delta_stats(res: Dict[str, Any], metric: str = "f1") -> Dict[str, Any]:
29         """Paired (by video) B?A summary for one metric: mean delta + bootstrap CI + sign
30         test
31         (eval_stats, C1 ? "never a bare mean"). Aligns the two variants' per-video held-out
32         scores
33         by video name, so the headline ?F1 comes with how-wide + did-it-win-on-enough-
34         videos."""
35         by_a = {p["video"]: p[metric] for p in res["variant_a"]["per_video"]}
36         by_b = {p["video"]: p[metric] for p in res["variant_b"]["per_video"]}
37         vids = [p["video"] for p in res["variant_a"]["per_video"]]
38         return paired_delta_ci([by_a[v] for v in vids], [by_b[v] for v in vids])
39
40     def load_videos(features_dir: str) -> List[experiment.VideoCandidates]:
41         """Load every ``*_golden_features.json`` into a VideoCandidates (intervals + the 10
42         feature columns + golden gts). The harness derives labels from gts itself."""
43         videos: List[experiment.VideoCandidates] = []
44         for path in sorted(glob.glob(os.path.join(features_dir, "*_golden_features.json"))):
45             with open(path, encoding="utf-8") as f:
46                 d = json.load(f)
47                 cands = d["candidates"]
48                 intervals = [(float(c["start"]), float(c["end"])) for c in cands]
49                 features: Dict[str, List[float]] = {}
50                 for fn in SHUTTLE_FEATURE_NAMES:
51                     features[fn] = [float(c["shuttle"][fn]) for c in cands]
52                 for fn in PERSON_FEATURE_NAMES:
53                     features[fn] = [float(c["person"][fn]) for c in cands]
54                 gts = [(float(a), float(b)) for a, b in d["gts"]]
55                 videos.append(experiment.VideoCandidates(name=d["name"], intervals=intervals,
56                                                             features=features, gts=gts))
57
58     return videos
```

```

54
55     def compare(videos: List[experiment.VideoCandidates], iou: float = 0.5,
56                 threshold: float = 0.5) -> Dict[str, Any]:
57         shuttle_only = experiment.Variant(name="shuttle_only",
58         feature_names=list(SHUTTLE_FEATURE_NAMES),
59         threshold=threshold)
60         fusion = experiment.Variant(name="shuttle_person_fusion",
61         feature_names=list(SHUTTLE_FEATURE_NAMES) +
62         list(PERSON_FEATURE_NAMES),
63         threshold=threshold)
64         return experiment.compare(videos, shuttle_only, fusion, iou=iou, honest=True)
65
66     def format_result(res: Dict[str, Any], iou: float) -> str:
67         a, b, d = res["variant_a"]["aggregate"], res["variant_b"]["aggregate"], res["delta"]
68         ds = delta_stats(res)
69         st = ds["sign_test"]
70         lines = [
71             f"=== Fusion P3 LOVO (n={res['num_videos']} golden videos, IoU>={iou}, honest)
72             ===",
73             f" shuttle-only : F1={a['f1']:.3f} P={a['precision']:.3f} R={a['recall']:.3f}
74             IoU={a['mean_iou']:.3f}",
75             f" +person : F1={b['f1']:.3f} P={b['precision']:.3f} R={b['recall']:.3f}
76             IoU={b['mean_iou']:.3f}",
77             f" ?F1 = {d['f1']:+.3f} ({res['delta_pct']['f1']:+.1f}%)
78             ?P={d['precision']:+.3f} ?R={d['recall']:+.3f}",
79             f" paired ?F1 {ds['mean_delta']:+.3f} [95% CI {ds['ci_low']:+.3f},
80             {ds['ci_high']:+.3f}] "
81             f"| sign test {int(st['wins'])}W/{int(st['losses'])}L (p={st['p_value']:.3f}) "
82             f"| CI clears 0: {ds['ci_excludes_zero']}",
83             f" per-video ?F1:",
84             ]
85         for pv in res["per_video_delta"]:
86             lines.append(f" {pv['video']:<26s} ?F1={pv['f1']:+.3f}")
87         return "\n".join(lines)
88
89     def main() -> None:
90         ap = argparse.ArgumentParser(description="LOVO fusion lift: shuttle-only vs
91         shuttle+person (golden set).")
92         ap.add_argument("--features-dir", default="output")
93         ap.add_argument("--iou", type=float, default=0.5)
94         ap.add_argument("--record", action="store_true", help="append the result to the
95         experiment leaderboard")
96         args = ap.parse_args()
97
98         videos = load_videos(args.features_dir)
99         if len(videos) < 2:
100             raise SystemExit(f"need >=2 *_golden_features.json in {args.features_dir} (found
101             {len(videos)})")
102         res = compare(videos, iou=args.iou)
103         print(format_result(res, args.iou))
104         if args.record:
105             row = experiment.record_experiment(res)
106             print(f"recorded -> eval_baselines/experiment_leaderboard.json
107             ({row['timestamp']}")
108
109 if __name__ == "__main__":
110     main()
111

```

19. user (2026-06-14T10:36:54.337Z)

```

1     """GPU-free tests for the LOSO/LOGO ablation runner (`backend.eval.ablation`).

```

```

2
3     Covers, all on deterministic synthetic candidate rows (no GPU, no real video):
4
5     - LOSO / LOGO correctness ? a cleanly-separating signal *earns its keep*, a noise
signal is
6     *inert*; LOGO collapses a group's columns; the ablated variant subtracts columns /
zeroes
7     the gate knob without mutating ``full``.
8     - the structural-guard exemption ? the SAME signal marked ``structural=True`` is
reported
9     (honest ?) but verdict is always ``structural_protected``, never auto-demoted on "no
lift".
10    - the honest-stats contract ? every row ships a CI + sign test, and no verdict is
stronger
11    than the stats support (the no-over-claim contract).
12    - a valid non-trivial PDF is produced (asserted via pypdf page count > 0).
13    - the golden litmus reproduction ? on the 6 real ``output/*_golden_features.json`` the
runner
14    reproduces the hand-measured Gate-A result (?F1 ? +0.074, 6/6 wins, p ? 0.031).
Skips
15    cleanly when the golden JSONs are absent (CI), runs when present (owner box).
16    ""
17    from __future__ import annotations
18
19    import glob
20    import json
21    import os
22
23    import pytest
24
25    from backend.eval import ablation
26    from backend.eval.ablation import (
27        AblationConfig,
28        AblationReport,
29        Signal,
30        ablate_one,
31        run_ablation,
32    )
33    from backend.eval.experiment import Variant, VideoCandidates
34
35
36    # ----- synthetic
fixtures
37
38
39    def _gate_videos(n: int = 6, *, separating: bool = True) -> list[VideoCandidates]:
40        """``n`` videos with a ``net_crossings`` gate signal.
41
42        ``separating=True``: rallies have net_crossings=5, junk has 0 ? so a
``min_crossings=2`` gate
43        perfectly removes the junk and keeping it is load-bearing. ``separating=False``:
every window
44        has net_crossings=5, so the gate is a no-op (ablating it changes nothing ?
inert)."""
45        videos: list[VideoCandidates] = []
46        for k in range(n):
47            intervals = [(0.0, 10.0), (20.0, 30.0), (40.0, 41.0), (50.0, 51.0)]
48            junk_val = 5.0 if not separating else 0.0
49            feats = {"net_crossings": [5.0, 5.0, junk_val, junk_val]}
50            gts = [(0.0, 10.0), (20.0, 30.0)]
51            videos.append(VideoCandidates(name=f"v{k}", intervals=intervals, features=feats,
gts=gts))
52        return videos
53
54

```



```

55     def _learned_videos(n: int = 6) -> list[VideoCandidates]:
56         """`n` videos where a PERSON feature (`players_in_court`) cleanly separates
rallies from
57         junk while the shuttle/audio columns are constant (non-separating). So a learned
full-set
58         ablation must show person as the load-bearing group and the others as washes ? a
deterministic
59         LOGO/LOSO correctness fixture (mirrors `test_fusion_compare`'s construction)."""
60         videos: list[VideoCandidates] = []
61         for k in range(n):
62             intervals = [(0.0, 10.0), (20.0, 30.0), (40.0, 41.0), (50.0, 51.0)]
63             feats: dict[str, list[float]] = {}
64             for fn in ablation.SHUTTLE_COLUMNS:
65                 feats[fn] = [1.0, 1.0, 1.0, 1.0] # constant ? non-
separating
66             for fn in ablation.PERSON_COLUMNS:
67                 feats[fn] = [0.0, 0.0, 0.0, 0.0]
68             feats["players_in_court"] = [4.0, 4.0, 0.0, 0.0] # the separating signal
69             feats["in_court_ratio"] = [1.0, 1.0, 0.0, 0.0]
70             for fn in ablation.AUDIO_COLUMNS:
71                 feats[fn] = [0.0, 0.0, 0.0, 0.0]
72             gts = [(0.0, 10.0), (20.0, 30.0)]
73             videos.append(VideoCandidates(name=f"v{k}", intervals=intervals, features=feats,
gts=gts))
74         return videos
75
76
77         # ----- LOSO
correctness
78
79
80     def test_separating_gate_earns_keep():
81         """A non-structural gate that cleanly separates rallies must EARN ITS KEEP: removing
it
82         drops F1 with the ?F1 CI clearing 0 and the sign test agreeing (6W/0L)."""
83         videos = _gate_videos(6, separating=True)
84         sig = Signal("gate_x", "gate_x", (), gate_knob="min_crossings", structural=False)
85         cfg = AblationConfig(full=Variant(name="full", min_crossings=2), signals=[sig],
granularity="signal")
86         row = ablate_one(videos, cfg, sig)
87         assert row.delta_f1 > 0.0
88         assert row.ci_excludes_zero is True
89         assert row.sign_wins == 6 and row.sign_losses == 0
90         assert row.sign_p_value <= 0.05
91         assert row.verdict == ablation.EARNS_KEEP
92         assert row.structural_protected is False
93         # C4: the gate cuts over-segmentation (more junk segments without it).
94         assert row.seg_ratio_full < row.seg_ratio_ablated
95
96
97
98     def test_noop_gate_is_inert():
99         """A gate whose threshold removes nothing (all windows pass) is a wash ? INERT,
never a
100         stronger claim. Honest: a single wash at n=6 is a candidate, not 'proven
useless'."""
101         videos = _gate_videos(6, separating=False)
102         sig = Signal("gate_x", "gate_x", (), gate_knob="min_crossings", structural=False)
103         cfg = AblationConfig(full=Variant(name="full", min_crossings=2), signals=[sig],
granularity="signal")
104         row = ablate_one(videos, cfg, sig)
105         assert abs(row.delta_f1) < 1e-9
106         assert row.ci_excludes_zero is False
107         assert row.verdict == ablation.INERT
108
109
110

```

```

111     def test_ablated_variant_does_not_mutate_full():
112         """Building the ablated variant must derive a new Variant, never mutate ``full`` in
place."""
113         full = Variant(name="full", feature_names=["a", "b", "c"], min_crossings=2)
114         before = full.to_dict()
115         ablated = ablation._ablated_variant(full, ("b",), "min_crossings")
116         assert ablated.feature_names == ["a", "c"]
117         assert ablated.min_crossings == 0
118         assert full.to_dict() == before          # full untouched
119         assert full.feature_names == ["a", "b", "c"]
120
121
122     def test_unknown_gate_knob_raises():
123         full = Variant(name="full", feature_names=["a"])
124         with pytest.raises(ValueError):
125             ablation._ablated_variant(full, (), "min_bogus")
126
127
128     # ----- LOGO
correctness
129
130
131     def test_logo_collapses_groups():
132         """LOGO (``granularity='group'``) ablates a whole producer's columns at once: the
default
133         5-signal LOSO inventory collapses to 4 producer groups, and the union of columns is
dropped."""
134         groups = ablation._collapse_to_groups(ablation.default_signals())
135         names = [g.name for g in groups]
136         assert names == ["shuttle", "gate_a", "person", "audio"]
137         shuttle = next(g for g in groups if g.name == "shuttle")
138         # the shuttle group unions duration + the velocity block
139         assert set(shuttle.columns) == {"duration", "peak_velocity", "mean_velocity",
"active_ratio"}
140         gate = next(g for g in groups if g.name == "gate_a")
141         assert gate.gate_knob == "min_crossings"
142         assert gate.structural is True          # structural flag OR-ed up
143
144
145     def test_logo_finds_separating_group():
146         """On the learned fixture where PERSON cleanly separates, the LOGO person row must
be the
147         load-bearing one (largest positive marginal ?F1) and the constant groups must be
~washes."""
148         videos = _learned_videos(6)
149         cfg = AblationConfig(full=ablation.learned_full_variant(),
signals=ablation.default_signals(),
150                             granularity="group")
151         report = run_ablation(videos, cfg)
152         rows = {r.signal: r for r in report.rows}
153         assert set(rows) == {"shuttle", "gate_a", "person", "audio"}
154         top = report.sorted_rows()[0]
155         assert top.signal == "person"
156         assert rows["person"].delta_f1 > 0.0
157         # the constant audio group carries no information ? marginal ? 0
158         assert abs(rows["audio"].delta_f1) < 0.05
159
160
161     def test_run_ablation_both_granularities():
162         """LOSO yields one row per signal; LOGO yields one per group; both honest-LOVO on a
learned
163         full set."""
164         videos = _learned_videos(6)
165         loso = run_ablation(videos, AblationConfig(full=ablation.learned_full_variant(),
signals=ablation.default_signals(),
166

```

```

167                                     granularity="signal"))
168     logo = run_ablation(videos, AblationConfig(full=ablation.learned_full_variant(),
169                                     signals=ablation.default_signals(),
170                                     granularity="group"))
171     assert len(logo.rows) == len(ablation.default_signals()) # 5 LOSO rows
172     assert len(logo.rows) == 4 # 4 producer groups
173     assert logo.honest_lovo is True and logo.honest_lovo is True
174
175
176     def test_run_ablation_rejects_bad_granularity():
177         videos = _learned_videos(2)
178         cfg = AblationConfig(full=ablation.learned_full_variant(),
179                             signals=ablation.default_signals(),
180                             granularity="bogus")
181         with pytest.raises(ValueError):
182             run_ablation(videos, cfg)
183
184     def test_run_ablation_needs_a_video():
185         cfg = AblationConfig(full=ablation.learned_full_variant(),
186                             signals=ablation.default_signals())
187         from backend.eval.experiment import ExperimentError
188         with pytest.raises(ExperimentError):
189             run_ablation([], cfg)
190
191     # ----- structural-guard
192     exemption
193
194     def test_structural_guard_never_demoted():
195         """A structural guard with the SAME measured ? as a non-structural one that EARNS
196         its keep
197         must report verdict ``structural_protected`` (never auto-demoted), but record the
198         honest ?."""
199         videos = _gate_videos(6, separating=True)
200         non = Signal("gate_x", "gate_x", (), gate_knob="min_crossings", structural=False)
201         struct = Signal("gate_x", "gate_x", (), gate_knob="min_crossings", structural=True)
202         full = Variant(name="full", min_crossings=2)
203         cfg = AblationConfig(full=full, signals=[non], granularity="signal")
204         r_non = ablate_one(videos, cfg, non)
205         r_struct = ablate_one(videos, cfg, struct)
206         # identical honest measurement?
207         assert pytest.approx(r_non.delta_f1) == r_struct.delta_f1
208         assert (r_non.sign_wins, r_non.sign_losses) == (r_struct.sign_wins,
209         r_struct.sign_losses)
210         # ?but the verdict differs: non-structural earns keep, structural is protected.
211         assert r_non.verdict == ablation.EARNS_KEEP
212         assert r_struct.verdict == ablation.STRUCTURAL_PROTECTED
213         assert r_struct.structural_protected is True
214
215     def test_structural_guard_protected_even_when_inert():
216         """A structural guard that is a measured wash is STILL reported as
217         ``structural_protected``
218         (never flagged dead weight on 'no measured lift')."""
219         videos = _gate_videos(6, separating=False)
220         struct = Signal("gate_x", "gate_x", (), gate_knob="min_crossings", structural=True)
221         cfg = AblationConfig(full=Variant(name="full", min_crossings=2), signals=[struct],
222                             granularity="signal")
223         row = ablate_one(videos, cfg, struct)
224         assert abs(row.delta_f1) < 1e-9 # honestly a wash
225         assert row.verdict == ablation.STRUCTURAL_PROTECTED
226         assert row.structural_protected is True

```

```

225
226     def test_default_gate_a_is_structural():
227         """The shipped Gate-A signal must carry ``structural=True`` (the held-shuttle FP
guard)."""
228         sig = next(s for s in ablation.default_signals() if s.group == "gate_a")
229         assert sig.structural is True
230         assert sig.gate_knob == "min_crossings"
231
232
233     # ----- honest-
stats fields
234
235
236     def test_row_carries_full_honest_evidence():
237         """Every row ships the honest evidence: n, CI bounds, the sign test, and a verdict
in the
238         closed vocabulary (no over-claim)."""
239         videos = _learned_videos(6)
240         cfg = AblationConfig(full=ablation.learned_full_variant(),
signals=ablation.default_signals(),
241                             granularity="group")
242         report = run_ablation(videos, cfg)
243         valid = {ablation.EARNS_KEEP, ablation.SUGGESTIVE, ablation.INERT,
244                 ablation.STRUCTURAL_PROTECTED}
245         for r in report.rows:
246             d = r.as_dict()
247             assert {"delta_f1", "ci_low", "ci_high", "ci_excludes_zero", "sign_test", "n",
248                     "verdict"} <= set(d)
249             assert d["n"] == 6
250             assert d["sign_test"]["wins"] + d["sign_test"]["losses"] <= 6
251             # CI brackets the mean ( $\pm$ fp) ? never a bare mean detached from its interval.
252             assert r.ci_low - 1e-9 <= r.delta_f1 <= r.ci_high + 1e-9
253             assert r.verdict in valid
254
255
256     def test_classify_verdicts_cover_all_branches():
257         """``_classify`` maps evidence ? the closed verdict vocabulary, honestly:
258         structural always protected; CI-clears-0 + sign-agrees ? earns_keep; leans-positive
but
259         CI-spans-0 ? suggestive; a wash ? inert."""
260         # structural always protected, regardless of evidence
261         assert ablation._classify(True, True, 6, 0, 0.01, 0.05) ==
(ablation.STRUCTURAL_PROTECTED, True)
262         assert ablation._classify(True, False, 0, 6, 1.0, 0.05) ==
(ablation.STRUCTURAL_PROTECTED, True)
263         # earns keep: CI clears 0 AND sign agrees
264         assert ablation._classify(False, True, 6, 0, 0.03, 0.05) == (ablation.EARNS_KEEP,
False)
265         # suggestive: leans positive (more wins) but CI spans 0 / not significant
266         assert ablation._classify(False, False, 4, 2, 0.30, 0.05) == (ablation.SUGGESTIVE,
False)
267         # inert: not more wins than losses and CI spans 0
268         assert ablation._classify(False, False, 2, 4, 0.69, 0.05) == (ablation.INERT, False)
269         assert ablation._classify(False, False, 2, 2, 1.0, 0.05) == (ablation.INERT, False)
270
271
272     def test_no_over_claim_vocabulary():
273         """The verdict vocabulary must NOT contain 'irrelevant'/'useless' (attribution
contract)."""
274         forbidden = {"irrelevant", "useless", "dead", "remove"}
275         for v in ablation.VERDICT_LABEL.values():
276             assert not any(bad in v.lower() for bad in forbidden)
277
278
279     def test_report_round_trips_json():

```

```

280         """The structured report is JSON-able and sorted by ?F1 desc (contribution
ranking)."""
281     videos = _learned_videos(6)
282     report = run_ablation(videos, AblationConfig(full=ablation.learned_full_variant(),
283                                                  signals=ablation.default_signals(),
284                                                  granularity="group"))
285     blob = json.dumps(report.as_dict()) # must not raise
286     back = json.loads(blob)
287     deltas = [r["delta_f1"] for r in back["rows"]]
288     assert deltas == sorted(deltas, reverse=True) # sorted desc
289     assert back["num_videos"] == 6
290     assert back["label_set_hash"] # provenance present
291
292
293     def test_seed_is_deterministic():
294         """Two runs with the same seed produce identical CIs (deterministic bootstrap)."""
295         videos = _learned_videos(6)
296         cfg = AblationConfig(full=ablation.learned_full_variant(),
signals=ablation.default_signals(),
297                             granularity="group", seed=7)
298         r1 = run_ablation(videos, cfg)
299         r2 = run_ablation(videos, cfg)
300         assert [r.ci_low for r in r1.rows] == [r.ci_low for r in r2.rows]
301         assert [r.ci_high for r in r1.rows] == [r.ci_high for r in r2.rows]
302
303
304     # ----- table +
PDF
305
306
307     def test_format_table_is_sorted_and_readable():
308         videos = _learned_videos(6)
309         report = run_ablation(videos, AblationConfig(full=ablation.learned_full_variant(),
310                                                  signals=ablation.default_signals(),
311                                                  granularity="group"))
312         out = ablation.format_table(report)
313         assert "Ablation report" in out
314         assert "?F1(full?ablated)" in out
315         assert "verdict" in out
316         # the highest-?F1 signal name appears before the lowest in the rendered order
317         top = report.sorted_rows()[0].signal
318         bottom = report.sorted_rows()[-1].signal
319         assert out.index(top) < out.index(bottom)
320
321
322     def test_export_pdf_is_valid_nontrivial(tmp_path):
323         """A valid PDF with at least one page is produced, and it contains the verdict
legend."""
324         pytest.importorskip("reportlab")
325         pypdf = pytest.importorskip("pypdf")
326         videos = _learned_videos(6)
327         report = run_ablation(videos, AblationConfig(full=ablation.learned_full_variant(),
328                                                  signals=ablation.default_signals(),
329                                                  granularity="group"))
330         out = str(tmp_path / "ablation.pdf")
331         ablation.export_pdf(report, out)
332         assert os.path.isfile(out)
333         assert os.path.getsize(out) > 1000 # non-trivial
334         reader = pypdf.PdfReader(out)
335         assert len(reader.pages) > 0
336         text = "".join(p.extract_text() for p in reader.pages)
337         assert "Ablation" in text
338         assert "clears-bar" in text # the verdict key rendered
339
340

```

```

341 # ----- corpus loader
(no GPU)
342
343
344 def test_load_videos_tolerates_missing_audio(tmp_path):
345     """An older feature JSON without an ``audio`` block still loads ? audio columns
default to
346     0.0 (loader never crashes on a pre-audio JSON)."""
347     cand = {"start": 0.0, "end": 10.0,
348             "shuttle": {fn: 1.0 for fn in ablation.SHUTTLE_COLUMNS},
349             "person": {fn: 0.0 for fn in ablation.PERSON_COLUMNS},
350             "label": 1}
351     data = {"name": "noaudio", "fps": 30.0, "frame_width": 1920.0,
352            "candidates": [cand], "gts": [[0.0, 10.0]]}
353     with open(tmp_path / "noaudio_golden_features.json", "w", encoding="utf-8") as f:
354         json.dump(data, f)
355     videos = ablation.load_videos(str(tmp_path))
356     assert len(videos) == 1
357     assert videos[0].features["audio_hit_count"] == [0.0] # defaulted, not
crashed
358
359
360 # ----- the
golden litmus
361
362     GOLDEN_DIR = os.path.join(
363         os.path.dirname(os.path.abspath(__file__)), "output")
364
365
366     def _golden_present() -> bool:
367         return len(glob.glob(os.path.join(GOLDEN_DIR, "*_golden_features.json"))) >= 6
368
369
370     @pytest.mark.skipif(not _golden_present(),
371                         reason="golden output/*_golden_features.json absent (run
fusion_golden first)")
372     def test_litmus_reproduces_gate_a():
373         """LITMUS: on the 6 real golden feature JSONs the runner reproduces the hand-
measured Gate-A
374         result ? marginal ?F1 ? +0.074, 6/6 wins, p ? 0.031, CI ? [+0.042, +0.104], within
tolerance.
375         This validates the whole framework against the recorded ground truth. Skips cleanly
in CI
376         where the golden substrate is absent; runs on the owner box where it is present."""
377         videos = ablation.load_videos(GOLDEN_DIR)
378         assert len(videos) == 6
379         cfg = ablation.litmus_config()
380         report = run_ablation(videos, cfg)
381         assert len(report.rows) == 1
382         row = report.rows[0]
383         assert row.signal == "gate_a"
384         # The recorded QUALITY_ITERATIONS truth (2026-06-14), reproduced within tolerance.
385         assert row.delta_f1 == pytest.approx(0.074, abs=0.005)
386         assert row.sign_wins == 6 and row.sign_losses == 0
387         assert row.sign_p_value == pytest.approx(0.031, abs=0.005)
388         assert row.ci_low == pytest.approx(0.042, abs=0.01)
389         assert row.ci_high == pytest.approx(0.104, abs=0.01)
390         assert row.ci_excludes_zero is True
391         # Gate-A is structural ? reported but never auto-promoted to earns_keep.
392         assert row.verdict == ablation.STRUCTURAL_PROTECTED
393         # C4: over-segmentation drops 1.68 ? 1.01 when Gate-A is on (full).
394         assert row.seg_ratio_full == pytest.approx(1.01, abs=0.05)
395         assert row.seg_ratio_ablated == pytest.approx(1.68, abs=0.05)
396
397

```

```

398     @pytest.mark.skipif(not _golden_present(),
399                           reason="golden output/*_golden_features.json absent")
400     def test_litmus_report_is_well_formed():
401         """The litmus report is a proper AblationReport (provenance + a sorted single
row)."""
402         report = run_ablation(ablation.load_videos(GOLDEN_DIR), ablation.litmus_config())
403         assert isinstance(report, AblationReport)
404         assert report.num_videos == 6
405         assert report.full_set_signals == ["gate_a"]
406         assert report.label_set_hash
407         out = ablation.format_table(report)
408         assert "gate_a" in out
409
410
411     # ----- edge
cases
412
413
414     def test_min_players_gate_knob_zeroed():
415         """The ``min_players`` gate knob is zeroed on ablation (the Gate-B analog)."""
416         full = Variant(name="full", min_players=2, feature_names=["a"])
417         ablated = ablation._ablated_variant(full, (), "min_players")
418         assert ablated.min_players == 0
419         assert ablated.feature_names == ["a"]          # columns untouched
420
421
422     def test_ablated_label_falls_back_to_gate_knob():
423         """With no columns, the ablated variant name carries the gate-knob label, not
'?'."""
424         full = Variant(name="full", min_crossings=2)
425         ablated = ablation._ablated_variant(full, (), "min_crossings")
426         assert "min_crossings" in ablated.name
427
428
429     # -----
CLI
430
431
432     def _write_learned_corpus(d: str, n: int = 6) -> None:
433         for k in range(n):
434             cands = []
435             for (s, e), pos in [((0.0, 10.0), 1), ((20.0, 30.0), 1), ((40.0, 41.0), 0),
((50.0, 51.0), 0)]:
436                 shuttle = {fn: 1.0 for fn in ablation.SHUTTLE_COLUMNS}
437                 person = {fn: 0.0 for fn in ablation.PERSON_COLUMNS}
438                 person["players_in_court"] = 4.0 if pos else 0.0
439                 person["in_court_ratio"] = 1.0 if pos else 0.0
440                 audio = {fn: 0.0 for fn in ablation.AUDIO_COLUMNS}
441                 cands.append({"start": s, "end": e, "shuttle": shuttle, "person": person,
442                             "audio": audio, "label": pos})
443                 data = {"name": f"v{k}", "fps": 30.0, "frame_width": 1920.0,
444                         "candidates": cands, "gts": [[0.0, 10.0], [20.0, 30.0]]}
445                 with open(os.path.join(d, f"v{k}_golden_features.json"), "w", encoding="utf-8")
as f:
446                     json.dump(data, f)
447
448
449     def test_cli_main_writes_pdf_and_json(tmp_path, monkeypatch, capsys):
450         """``main()`` runs the LOGO ablation and writes both a PDF and the structured
JSON."""
451         pytest.importorskip("reportlab")
452         pytest.importorskip("pypdf")
453         d = str(tmp_path / "feat")
454         os.makedirs(d)
455         _write_learned_corpus(d)

```

```

456     pdf = str(tmp_path / "out.pdf")
457     js = str(tmp_path / "out.json")
458     monkeypatch.setattr("sys.argv", ["ablation", "--features-dir", d, "--granularity",
"group",
459                                     "--pdf", pdf, "--json", js])
460     ablation.main()
461     out = capsys.readouterr().out
462     assert "Ablation report" in out
463     assert os.path.isfile(pdf) and os.path.getsize(pdf) > 1000
464     assert os.path.isfile(js)
465     report = json.loads(open(js, encoding="utf-8").read())
466     assert {r["group"] for r in report["rows"]} == {"shuttle", "gate_a", "person",
"audio"}
467
468
469     def test_cli_main_litmus_flag(tmp_path, monkeypatch, capsys):
470         """`--litmus` runs the static Gate-A config (single structural row)."""
471         d = str(tmp_path / "feat")
472         os.makedirs(d)
473         _write_learned_corpus(d)
474         monkeypatch.setattr("sys.argv", ["ablation", "--features-dir", d, "--litmus"])
475         ablation.main()
476         out = capsys.readouterr().out
477         assert "gate_a" in out
478         assert "static" in out                                     # litmus full set is the static
gate variant
479
480
481     def test_cli_main_needs_two_videos(tmp_path, monkeypatch):
482         d = str(tmp_path / "feat")
483         os.makedirs(d)
484         _write_learned_corpus(d, n=1)
485         monkeypatch.setattr("sys.argv", ["ablation", "--features-dir", d])
486         with pytest.raises(SystemExit):
487             ablation.main()
488

```

20. assistant (2026-06-14T10:37:01.522Z)

I now understand the offline ablation litmus deeply. The crucial insight: the offline ablation/``fusion_compare`/`experiment`` path builds candidates and gating with ``rally_gate.gate_windows`` over candidate windows built by ``distill_local.build_candidates``. The SERVED path is ``FusionSegmenter._select_for_handoff`` over windows from ``TrajectoryHybridSegmenter`'s``trajectory_to_action_windows`, enriched by `_enrich_candidate_stats` (which calls `rally_gate.gate_windows`).`

The task wants me to run the **production** ``FusionSegmenter`` selection logic (not the offline experiment harness) on cached trajectories. The key difference: the served path uses ``trajectory_to_action_windows`` (the actual windowing the production segmenter uses) + ``_enrich_candidate_stats`` net crossings + ``_select_for_handoff`` gating, then scores the resulting kept windows vs golden GTs with ``metrics.score``.

Let me check ``distill_local.build_candidates``, ``trajectory_to_action_windows``, and how the experiment harness builds windows, to understand the divergence between offline and served candidate-building.

21. user (2026-06-14T10:37:02.942Z)

```

1     """Distill Gemini (teacher) into a LEARNED local rally classifier (student).
2
3     Threshold-tuning the local knobs hit a low ceiling (see calibrate_local). This goes
4     further: learn "is this WASB candidate a real rally?" from Gemini silver labels,
5     using a few resolution/fps-INVARIANT local features, so the model transfers across
6     videos and runs 100% local at inference (no Gemini, privacy lever intact).
7

```



```

8     Flow (all local at runtime):
9     WASB trajectory -> recall-oriented candidate windows (propose many; classifier
10    filters) -> per-candidate features [duration, peak_velocity, mean_velocity,
11    active_ratio, net_crossings] -> LogisticRegression(prob >= threshold) -> merge
12    -> rally windows.
13
14    Training labels come from Gemini full-context labels via the SAME IoU matching the
15    evaluator uses (training.label_candidates). Honest leave-one-video-out: train on the
16    other videos' candidates, predict the held-out one, score vs its Gemini labels, and
17    compare against the threshold baseline. Reuses classifier.py, training.label_candidates,
18    rally_gate, metrics, calibrate_wasb.load_golden_gts, gemini_refine.merge_overlaps.
19
20    Run:
21    python -m backend.eval.distill_local --manifest videos.json --model-out
output/local_rally_model.json
22    """
23    from __future__ import annotations
24
25    import json
26    import argparse
27    from typing import Dict, List, Optional, Tuple
28
29
30    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
31    from backend.pipeline.detectors import rally_gate
32    from backend.eval.calibrate_wasb import load_golden_gts
33    from backend.eval.training import label_candidates
34    from backend.eval.classifier import LogisticRegression
35    from backend.eval.metrics import score
36    from backend.eval.gemini_refine import merge_overlaps
37
38    Interval = Tuple[float, float]
39
40    FEATURE_NAMES = ["duration", "peak_velocity", "mean_velocity", "active_ratio",
"net_crossings"]
41    # Recall-oriented proposal: deliberately permissive so real rallies are among the
42    # candidates (the classifier removes the junk). (velocity_thresh, merge_gap, min_dur)
43    PROPOSAL = (0.005, 1.0, 0.5)
44    BASELINE_LOCAL = (0.02, 2.0, 2.0) # today's production-ish windowing, no learning
45
46
47    def build_candidates(trajectory_csv: str, fps: float, frame_width: float,
48    proposal: Tuple[float, float, float] = PROPOSAL) ->
Tuple[List[Interval], List[List[float]]]:
49    """WASB trajectory -> (candidate intervals, fps/resolution-invariant feature
rows)."""
50    points = TrackNetRunner.parse_trajectory_csv(trajectory_csv)
51    vt, mg, mwd = proposal
52    wins = TrackNetRunner.trajectory_to_action_windows(
53    points, fps=fps, frame_width=frame_width,
54    velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
55    intervals = [(w["start"], w["end"]) for w in wins]
56    gated = rally_gate.gate_windows(points, intervals, fps, min_crossings=0) # for net-
crossings
57    X: List[List[float]] = []
58    for w, g in zip(wins, gated):
59    dur = max(1e-6, w["end"] - w["start"])
60    win_frames = max(1.0, dur * fps)
61    X.append([
62    dur, # rally length (sec) ?
63    float(w["peak_velocity"]), # already normalized by
64    float(w["mean_velocity"]),
65    float(w["active_frame_count"]) / win_frames, # active-shuttle ratio ?

```

```

fps-invariant
66         float(g.crossings),                                # net crossings (count) ?
the rally signal
67     ])
68     return intervals, X
69
70
71     def baseline_windows(trajecory_csv: str, fps: float, frame_width: float) ->
List[Interval]:
72         points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
73         vt, mg, mwd = BASELINE_LOCAL
74         wins = TrackNetRunner.trajectory_to_action_windows(
75             points, fps=fps, frame_width=frame_width,
76             velocity_thresh=vt, merge_gap=mg, min_window_duration=mwd)
77         return [(w["start"], w["end"]) for w in wins]
78
79
80     def load_video(spec: Dict, iou: float = 0.5) -> Dict:
81         fps, fw = float(spec["fps"]), float(spec["frame_width"])
82         intervals, X = build_candidates(spec["trajectory"], fps, fw)
83         gts = load_golden_gts(spec["labels"])
84         y = label_candidates(intervals, gts, iou)
85         return {"name": spec.get("name", spec["trajectory"]), "intervals": intervals, "X":
X,
86             "y": y, "gts": gts, "baseline": baseline_windows(spec["trajectory"], fps,
fw)}
87
88
89     def predict_rallies(model: LogisticRegression, X: List[List[float]], intervals:
List[Interval],
90         threshold: float = 0.5) -> List[Interval]:
91         if not intervals:
92             return []
93         proba = model.predict_proba(X)
94         pos = [iv for iv, p in zip(intervals, proba) if p >= threshold]
95         return merge_overlaps(pos, gap_tol=1.0)
96
97
98     def run(specs: List[Dict], iou: float = 0.5, threshold: float = 0.5,
99         model_out: Optional[str] = None) -> dict:
100         videos = [load_video(s, iou) for s in specs]
101         print(f"=== Distilled local rally classifier vs Gemini silver-GT "
102             f"({len(videos)} videos, IoU>={iou}, prob>={threshold}) ===")
103         for v in videos:
104             ceil = score(v["intervals"], v["gts"], iou).recall
105             print(f"[{v['name']}] {len(v['intervals'])} candidates ({sum(v['y'])} pos) | "
106                 f"proposal recall ceiling {ceil:.3f} | {len(v['gts'])} Gemini rallies")
107
108         result: dict = {"videos": [v["name"] for v in videos]}
109         if len(videos) >= 2:
110             print("\n--- HONEST leave-one-video-out: learned vs threshold baseline (held-
out) ---")
111             clf, base = [], []
112             for i, held in enumerate(videos):
113                 others = [v for j, v in enumerate(videos) if j != i]
114                 X = [row for v in others for row in v["X"]]
115                 y = [t for v in others for t in v["y"]]
116                 model = LogisticRegression().fit(X, y, FEATURE_NAMES)
117                 preds = predict_rallies(model, held["X"], held["intervals"], threshold)
118                 sc = score(preds, held["gts"], iou)
119                 bsc = score(held["baseline"], held["gts"], iou)
120                 clf.append(sc)
121                 base.append(bsc)
122                 print(f" held-out {held['name']}: LEARNED P={sc.precision:.3f}
R={sc.recall:.3f} F1={sc.f1:.3f}")

```

```

123         f" | baseline P={bsc.precision:.3f} R={bsc.recall:.3f}
F1={bsc.f1:.3f}")
124     def mean(xs, a):
125         return sum(getattr(s, a) for s in xs) / len(xs)
126     print(f" MEAN held-out: LEARNED P={mean(clf, 'precision'):.3f}
R={mean(clf, 'recall'):.3f} "
127           f"F1={mean(clf, 'f1'):.3f} | baseline P={mean(base, 'precision'):.3f} "
128           f"R={mean(base, 'recall'):.3f} F1={mean(base, 'f1'):.3f}")
129     result["mean_learned_f1"] = mean(clf, "f1")
130     result["mean_baseline_f1"] = mean(base, "f1")
131
132     # Final model on ALL videos (for production / running on a fresh held-out video).
133     Xall = [row for v in videos for row in v["X"]]
134     yall = [t for v in videos for t in v["y"]]
135     final = LogisticRegression().fit(Xall, yall, FEATURE_NAMES)
136     assert final.w is not None # set by fit()
137     weights = dict(zip(FEATURE_NAMES, [round(float(w), 3) for w in final.w]))
138     print(f"\nfinal model standardized weights (sign/size = influence): {weights}")
139     if model_out:
140         final.save(model_out)
141         print(f"final model (all videos) -> {model_out}")
142         result["model_out"] = model_out
143     return result
144
145
146     def main() -> None:
147         ap = argparse.ArgumentParser(description="Distill Gemini labels into a local rally
classifier.")
148         ap.add_argument("--manifest", required=True,
149                         help="JSON: [{name, trajectory, fps, frame_width, labels}, ...]
(labels = Gemini CSV)")
150         ap.add_argument("--iou", type=float, default=0.5)
151         ap.add_argument("--threshold", type=float, default=0.5, help="classifier probability
cutoff")
152         ap.add_argument("--model-out", default="output/local_rally_model.json")
153         args = ap.parse_args()
154         with open(args.manifest, encoding="utf-8") as f:
155             specs = json.load(f)
156         run(specs, iou=args.iou, threshold=args.threshold, model_out=args.model_out)
157
158
159     if __name__ == "__main__":
160         main()
161

```

22. user (2026-06-14T10:37:03.511Z)

```

1     """Experiment harness ? A/B + shadow + side-by-side for rally detection.
2
3     This is the thin **variant/experiment layer** that `docs/EXPERIMENT_HARNESS.md`
4     identifies as the one missing piece: ~80% of the machinery (scoring, LOVO, the
5     no-regression gate, the pinnable classifier artifact, the persisted-candidate
6     substrate) already exists ? this module composes it. **No new scoring math lives
7     here**; everything defers to:
8
9     - `backend.eval.metrics`           ? `score`, `match_intervals`, `interval_iou`
10    - `backend.eval.training`           ? `label_candidates` (y by the same IoU matcher)
11    - `backend.eval.classifier`         ? `LogisticRegression` (the pinnable JSON model)
12    - `backend.eval.gemini_refine`      ? `merge_overlaps` (the blessed window merge)
13    - `backend.eval.regression`         ? `gt_hash` (label-set fingerprint)
14    - `backend.eval.calibration`        ? `pct_improvement`
15
16    The thesis (`EXPERIMENT_HARNESS.md` §2): separate the **expensive, risky DETECTION**
17    (per-frame signals ? run once on the GPU/WSL box, persisted into
18    `candidate_segments.stats`) from the **cheap, swappable DECISION** (given those

```

```

19 signals, where are the rallies). A `Variant` is a declarative re-scoring recipe;
20 given persisted candidate features it produces `List[Interval]` deterministically,
21 with no GPU and zero production risk. So most experiments are pure offline
22 re-scoring of stored data and never touch the served pipeline.
23
24 Three modes (`EXPERIMENT_HARNESS.md` §5):
25 - `compare()` ? labeled A/B vs golden labels: per-video + LOVO-honest
26   aggregate deltas. Mirrors `distill_local`'s honest train-on-others/predict-
27   held-out loop, but variant-parameterised.
28 - `compare_unlabeled()` ? SxS on NEW footage with no ground truth: where do two
29   variants agree / diverge (A-only, B-only, agreed). The divergences are what a
30   human spot-checks (and what two highlight reels would show side by side).
31 - the promotion gate stays `run_regression.py` + `regression_baseline.json`
32   (exit 0/1/3); **rollback = flip the variant/config, never `git revert`**
33
34 Pure + offline + unit-tested. The only DB-touching helper is
35 `load_video_candidates`, which reads persisted candidates via
36 `Database.query_candidate_segments`.
37 """
38 from __future__ import annotations
39
40 import json
41 import logging
42 import os
43 from dataclasses import dataclass, field
44 from datetime import datetime, timezone
45 from hashlib import sha1
46 from typing import Any, Callable, Dict, List, Optional, Sequence
47
48 from backend.eval.calibration import pct_improvement
49 from backend.eval.classifier import LogisticRegression
50 from backend.eval.gemini_refine import merge_overlaps
51 from backend.eval.metrics import Interval, match_intervals, score
52 from backend.eval.regression import gt_hash
53 from backend.eval.training import label_candidates
54 from backend.eval import eval_stats as _stats # C1: CIs + paired sign test
55 from backend.eval import segmentation_metrics as _segm # C4: F1@k + segment-count
56 ratio
57 scores
58 from backend.eval.score_calibration import fit_isotonic, fit_platt # C2: calibrate cue
59
60 logger = logging.getLogger(__name__)
61
62 # Where a comparison run appends one reproducible record. Tracked location (NOT under
63 # the gitignored output/) so the leaderboard survives a clean checkout, mirroring
64 # regression.DEFAULT_BASELINE_PATH.
65 DEFAULT_LEADERBOARD_PATH = os.path.join("eval_baselines", "experiment_leaderboard.json")
66 _METRICS = ("precision", "recall", "f1", "mean_iou")
67
68 class ExperimentError(ValueError):
69     """A variant cannot be evaluated as configured (e.g. a learned variant asked to
70     score an unlabeled video with no pinned model, or a gate referencing an absent
71     feature)."""
72
73 # ----- Variant
74
75 @dataclass
76 class Variant:
77     """A declarative re-scoring recipe ? NOT a code branch (`EXPERIMENT_HARNESS.md` §4).
78
79     A variant turns one video's persisted candidate windows + features into rally
80     intervals. Every knob defaults to the **control** behaviour (no gate, no learned

```

```

82 filter), so a variant that merely *carries* a new, disabled cue is byte-identical
83 to control ? the core no-regression guarantee.
84
85 Fields:
86 - ``segmenter`` / ``config_overrides`` / ``cue_flags`` ? informational provenance
87   for the leaderboard and for selecting the served path (the existing
88   `segmenter_registry` + `IndexingConfig` do the actual selection in production).
89   New cues are recorded here default-OFF.
90 - ``min_crossings`` ? decision-gate **Gate A**: keep only windows whose persisted
91   ``net_crossings`` feature is >= this. 0 = off. This is the eval-only gate from
92   `rally_gate.py` expressed as a re-scoring knob (kills service-feed / held-
shuttle
93   windows ? see `MULTI_SIGNAL_FUSION_PLAN.md` §3.1).
94 - ``min_players`` ? decision-gate **Gate B** (the future person cue): keep windows
95   whose persisted ``players_in_court`` is >= this. 0 = off (no-op until the person
96   cue persists that feature).
97 - ``classifier_model`` ? path to a frozen `LogisticRegression` JSON. When set,
98   `compare()` scores videos with the SHIPPED model as-is (no per-fold training);
99   required for `compare_unlabeled` on a learned variant.
100 - ``feature_names`` ? the persisted features the learned filter consumes. When set
101   WITHOUT ``classifier_model``, `compare()` trains a fresh model per LOVO fold
102   (honest) ? the recipe, not a pinned artifact.
103 - ``threshold`` ? classifier probability cutoff. ``merge_gap`` ? post-filter
104   overlap-merge gap (seconds), the same `gemini_refine.merge_overlaps` default.
105 """
106
107 name: str
108 segmenter: str = "wasb_hybrid"
109 config_overrides: Dict[str, Any] = field(default_factory=dict)
110 cue_flags: Dict[str, bool] = field(default_factory=dict)
111 min_crossings: int = 0
112 min_players: int = 0
113 classifier_model: Optional[str] = None
114 feature_names: Optional[List[str]] = None
115 threshold: float = 0.5
116 merge_gap: float = 1.0
117 # C2 / threshold-in-LOVO (default OFF ? unchanged behaviour). When set, the
operating
118 # point is chosen on the TRAINING fold, never the held-out one ? fixing the
first-A/B
119 # failure where a fixed 0.5 zeroed out a small-candidate video.
120 tune_threshold: bool = False # pick the F1-best threshold on the train fold
121 calibrate: Optional[str] = None # None | "platt" | "isotonic" ? calibrate
proba first
122
123 def is_learned(self) -> bool:
124     """True if this variant applies a learned classifier (pinned model or
recipe)."""
125     return self.classifier_model is not None or bool(self.feature_names)
126
127 def to_dict(self) -> dict:
128     return {
129         "name": self.name,
130         "segmenter": self.segmenter,
131         "config_overrides": self.config_overrides,
132         "cue_flags": self.cue_flags,
133         "min_crossings": self.min_crossings,
134         "min_players": self.min_players,
135         "classifier_model": self.classifier_model,
136         "feature_names": self.feature_names,
137         "threshold": self.threshold,
138         "merge_gap": self.merge_gap,
139         "tune_threshold": self.tune_threshold,
140         "calibrate": self.calibrate,
141     }

```

```

142
143 @classmethod
144 def from_dict(cls, d: dict) -> "Variant":
145     known = {f for f in cls.__dataclass_fields__} # type: ignore[attr-defined]
146     return cls(**{k: v for k, v in d.items() if k in known})
147
148 def spec_hash(self) -> str:
149     """Stable 12-char hash of the recipe ? identifies the variant in the leaderboard
150     and makes a result reproducible. The pinned **control** variant always hashes
151     the
152     same, so a regression is always traceable to a recipe change, never to drift."""
153     blob = json.dumps(self.to_dict(), sort_keys=True, default=str)
154     return sha1(blob.encode()).hexdigest()[:12]
155
156 #: The pinned, frozen baseline: candidates as detected, merged, no gate, no learned
157 #: filter. Always the comparison reference; "rollback" = make this the served variant.
158 CONTROL = Variant(name="control")
159
160
161 # ----- persisted candidates
162
163
164 @dataclass
165 class VideoCandidates:
166     """One video's persisted detection, ready for offline re-scoring.
167
168     ``intervals`` are the candidate windows; ``features`` is column-major
169     (``feature_name`` -> per-candidate value, each list aligned to ``intervals``);
170     ``gts`` are golden labels (None for new/unlabeled footage). Build one from the DB
171     with ``load_video_candidates``, or directly in tests.
172     """
173
174     name: str
175     intervals: List[Interval]
176     features: Dict[str, List[float]] = field(default_factory=dict)
177     gts: Optional[List[Interval]] = None
178
179
180 def _feature_column(vc: VideoCandidates, name: str) -> List[float]:
181     """Persisted feature column, defaulting missing/short columns to 0.0 (matches
182     ``training.extract_yolo_features``, which lets a sparse/old row still vectorise)."""
183     col = vc.features.get(name)
184     n = len(vc.intervals)
185     if col is None:
186         logger.warning("variant references feature %r absent from %s ? treating as 0.0",
187                        name, vc.name)
188         return [0.0] * n
189     if len(col) != n:
190         raise ExperimentError(
191             f"feature {name!r} has {len(col)} values but {vc.name} has {n} candidates")
192     return [float(x) for x in col]
193
194
195 def _feature_matrix(vc: VideoCandidates, feature_names: Sequence[str]) ->
List[List[float]]:
196     cols = [_feature_column(vc, name) for name in feature_names]
197     return [list(row) for row in zip(*cols)] if cols else [[] for _ in vc.intervals]
198
199
200 def _gate_mask(variant: Variant, vc: VideoCandidates) -> List[bool]:
201     """Boolean keep-mask from the decision gates (Gate A net-crossings, Gate B
202     players)."""
203     n = len(vc.intervals)
204     mask = [True] * n

```

```

204     if variant.min_crossings > 0:
205         col = _feature_column(vc, "net_crossings")
206         mask = [m and (col[i] >= variant.min_crossings) for i, m in enumerate(mask)]
207     if variant.min_players > 0:
208         col = _feature_column(vc, "players_in_court")
209         mask = [m and (col[i] >= variant.min_players) for i, m in enumerate(mask)]
210     return mask
211
212
213     def predict(variant: Variant, vc: VideoCandidates,
214                 model: Optional[LogisticRegression] = None,
215                 threshold: Optional[float] = None,
216                 calibrator: Optional[Callable[[float], float]] = None) -> List[Interval]:
217         """Variant prediction: gate by persisted features, optionally filter by a learned
218         model, then merge. Pure and deterministic.
219
220         ``model`` (an already-fitted `LogisticRegression`) is supplied by `compare` per LOVO
221         fold; if omitted and the variant pins ``classifier_model``, that frozen model is
222         loaded. A learned variant with neither a supplied nor a pinned model raises ? there
223         is nothing to score with. ``threshold``/``calibrator`` (both fitted on the TRAINING
224         fold) override the variant defaults when the C2 / threshold-in-LOVO path supplies
225         them.
226
227         """
228         mask = _gate_mask(variant, vc)
229         if variant.feature_names:
230             if model is None:
231                 if variant.classifier_model is None:
232                     raise ExperimentError(
233                         f"variant {variant.name!r} is learned but has no model to predict "
234                         f"with (pass a fitted model or set classifier_model)")
235                 model = LogisticRegression.load(variant.classifier_model)
236                 proba = [float(p) for p in model.predict_proba(_feature_matrix(vc,
237 variant.feature_names))]
238             if calibrator is not None:
239                 proba = [calibrator(p) for p in proba]
240             thr = variant.threshold if threshold is None else threshold
241             mask = [m and (proba[i] >= thr) for i, m in enumerate(mask)]
242             kept = [iv for iv, keep in zip(vc.intervals, mask) if keep]
243             return merge_overlaps(kept, gap_tol=variant.merge_gap)
244
245     def _calibrate_and_tune(variant: Variant, model: LogisticRegression,
246                             train_videos: Sequence[VideoCandidates], iou: float
247                             ) -> "tuple[Optional[Callable[[float], float]],
248 Optional[float]]":
249         """Fit a calibrator and/or pick the operating threshold on the TRAINING fold only
250         (C2 + threshold-in-LOVO). Returns ``(calibrator, threshold)`` ? either may be None
251         (use the variant default). Honest: never looks at the held-out video.
252
253         """
254         calibrator: Optional[Callable[[float], float]] = None
255         if variant.calibrate:
256             raw: List[float] = []
257             y: List[int] = []
258             for vc in train_videos:
259                 raw.extend(float(p) for p in
260                             model.predict_proba(_feature_matrix(vc, variant.feature_names or
261 [])))
262                 y.extend(label_candidates(vc.intervals, vc.gts or [], iou))
263             if variant.calibrate == "platt":
264                 calibrator = fit_platt(raw, y)
265             elif variant.calibrate == "isotonic":
266                 calibrator = fit_isotonic(raw, y)
267             else:
268                 raise ExperimentError(f"unknown calibrate={variant.calibrate!r} (use
269 'platt'/'isotonic')")

```

```

264         threshold: Optional[float] = None
265         if variant.tune_threshold:
266             best_t, best_f1 = variant.threshold, -1.0
267             for k in range(1, 20): # grid 0.05..0.95 on the train
fold's rally F1
268                 t = round(0.05 * k, 2)
269                 f1s = [score(predict(variant, vc, model=model, threshold=t,
calibrator=calibrator),
270                             vc.gts or [], iou).f1 for vc in train_videos]
271                 mean_f1 = sum(f1s) / len(f1s) if f1s else 0.0
272                 if mean_f1 > best_f1:
273                     best_f1, best_t = mean_f1, t
274                 threshold = best_t
275             return calibrator, threshold
276
277
278         # ----- variant scoring
279
280
281     def _train(variant: Variant, videos: Sequence[VideoCandidates], iou: float) ->
LogisticRegression:
282         """Fit the variant's classifier on the pooled candidates of ``videos`` (labels via
the
283         evaluator's own IoU matcher). The honest-LOVO training step from `distill_local`. """
284         X: List[List[float]] = []
285         y: List[int] = []
286         for vc in videos:
287             if vc.gts is None:
288                 raise ExperimentError(f"cannot train: {vc.name} has no golden labels")
289             X.extend(_feature_matrix(vc, variant.feature_names or []))
290             y.extend(label_candidates(vc.intervals, vc.gts, iou))
291         if not X:
292             raise ExperimentError("cannot train a learned variant on zero candidates")
293         return LogisticRegression().fit(X, y, variant.feature_names)
294
295
296     def evaluate_variant(videos: Sequence[VideoCandidates], variant: Variant,
297                         iou: float = 0.5, honest: bool = True) -> Dict[str, Any]:
298         """Score a variant on a labeled corpus. Returns per-video Scores + predictions + a
299         mean aggregate.
300
301         Prediction policy:
302         - **static variant** (no learned filter): predict each video directly ? no
training,
303         so no leakage; per-video scoring is already honest.
304         - **learned, pinned model** (`classifier_model` set): score every video with the
305         SHIPPED model. ? optimistic if those videos were in its training set ? use this
to
306         report "how the shipped artifact does here", not for the promotion decision.
307         - **learned recipe** (`feature_names`, no pinned model) + ``honest=True``: LOVO
?
308         train on the OTHER videos, predict the held-out one. The number to trust.
309         - learned recipe + ``honest=False``: train on all, predict each (overfit; sanity
only).
310
311         """
312         for vc in videos:
313             if vc.gts is None:
314                 raise ExperimentError(f"{vc.name} has no golden labels; use
compare_unlabeled")
315
316         per_video: List[Dict[str, Any]] = []
317         predictions: Dict[str, List[Interval]] = {}
318
319         recipe_lovo = bool(variant.feature_names) and variant.classifier_model is None
pinned_model = (LogisticRegression.load(variant.classifier_model)

```



```

320             if variant.classifier_model is not None else None)
321     all_model = None
322     if recipe_lovo and not honest:
323         all_model = _train(variant, videos, iou)
324
325     for i, vc in enumerate(videos):
326         cal: Optional[Callable[[float], float]] = None
327         thr: Optional[float] = None
328         if recipe_lovo and honest:
329             others = [v for j, v in enumerate(videos) if j != i]
330             if not others:
331                 raise ExperimentError("LOVO needs >=2 videos; pass honest=False or a
pinned model")
332             model: Optional[LogisticRegression] = _train(variant, others, iou)
333             if variant.tune_threshold or variant.calibrate: # operating point from
the train fold
334                 assert model is not None # _train never returns
None
335                 cal, thr = _calibrate_and_tune(variant, model, others, iou)
336             elif recipe_lovo: # honest=False ? one model over all
videos
337                 model = all_model
338             else: # static variant or pinned model
339                 model = pinned_model
340             preds = predict(variant, vc, model=model, threshold=thr, calibrator=cal)
341             assert vc.gts is not None # guarded at function top
342             sc = score(preds, vc.gts, iou)
343             per_video.append({"video": vc.name, "num_pred": len(preds), **{m: getattr(sc, m)
for m in _METRICS}})
344             predictions[vc.name] = preds
345
346         aggregate = {m: (sum(p[m] for p in per_video) / len(per_video) if per_video else
0.0)
347                     for m in _METRICS}
348     return {
349         "variant": variant.name,
350         "spec_hash": variant.spec_hash(),
351         "is_learned": variant.is_learned(),
352         "honest_lovo": recipe_lovo and honest,
353         "per_video": per_video,
354         "aggregate": aggregate,
355         "predictions": predictions,
356     }
357
358
359     def _ci_block(eval_v: Dict[str, Any]) -> Dict[str, Any]:
360         """Per-metric mean + bootstrap CI over the per-video scores (C1 ? never a bare
mean)."""
361         return {m: _stats.mean_with_ci([p[m] for p in eval_v["per_video"]]) for m in
_METRICS}
362
363
364     def _segmentation_block(videos: Sequence[VideoCandidates], eval_v: Dict[str, Any]) ->
Dict[str, Any]:
365         """Mean F1@k + segment-count ratio across videos (C4 ? the over-segmentation tIoU
hides)."""
366         gts_by_name = {v.name: (v.gts or []) for v in videos}
367         overlaps = _segm.DEFAULT_OVERLAPS
368         f1k: Dict[float, List[float]] = {ov: [] for ov in overlaps}
369         ratios: List[float] = []
370         for name, preds in eval_v.get("predictions", {}).items():
371             gts = gts_by_name.get(name, [])
372             got = _segm.f1_at_overlaps(preds, gts, overlaps)
373             for ov in overlaps:
374                 f1k[ov].append(got[ov])

```

```

375         ratios.append(_segm.segment_count_ratio(preds, gts))
376
377     def _mean(xs: List[float]) -> float:
378         return sum(xs) / len(xs) if xs else 0.0
379     out: Dict[str, Any] = {f"f1@{int(ov * 100)}": _mean(vals) for ov, vals in
f1k.items()}
380     out["segment_count_ratio"] = _mean(ratios)
381     return out
382
383
384     def honest_summary(videos: Sequence[VideoCandidates], eval_a: Dict[str, Any],
385                       eval_b: Dict[str, Any]) -> Dict[str, Any]:
386         """C1 + C4 honest reporting layered over a compare() pair (additive,
EXPERIMENT_HARNESS §6).
387
388         ``per_video`` lists are video-aligned (``evaluate_variant`` iterates ``videos`` in
order),
389         so ``paired_delta_f1`` pairs B?A by video ? the promotion-grade view: a lift is real
when
390         the ?F1 CI clears 0 *and* the sign test agrees, not when a bare mean ticked up.
391         """
392         a_f1 = [p["f1"] for p in eval_a["per_video"]]
393         b_f1 = [p["f1"] for p in eval_b["per_video"]]
394         return {
395             "variant_a_ci": _ci_block(eval_a),
396             "variant_b_ci": _ci_block(eval_b),
397             "paired_delta_f1": _stats.paired_delta_ci(a_f1, b_f1),
398             "segmentation": {"variant_a": _segmentation_block(videos, eval_a),
399                             "variant_b": _segmentation_block(videos, eval_b)},
400         }
401
402
403     def compare(videos: Sequence[VideoCandidates], variant_a: Variant, variant_b: Variant,
404               iou: float = 0.5, honest: bool = True, honest_stats: bool = True) ->
Dict[str, Any]:
405         """Offline labeled A/B (`EXPERIMENT_HARNESS.md` §5.1). Scores both variants on the
same
406         golden corpus and reports the **B ? A** deltas, per video and in aggregate.
407
408         The deltas use the existing `metrics.score` (per video) +
`calibration.pct_improvement`;
409         learned variants are scored LOVO-honestly. The result is a self-contained record fit
for
410         `record_experiment` ? variant specs + hashes + the label-set fingerprint travel with
it.
411
412         ``honest_stats`` (default True) **adds** a ``"honest"`` block ? per-metric bootstrap
CIs, a
413         paired ?F1 CI + exact sign test (C1), and F1@k + segment-count ratio (C4) ?
*alongside* the
414         existing bare-mean fields (additive: nothing existing changes). Set False for the
legacy
415         shape. This is the measurement-honesty wiring (`ANALYZERS_AND_RECONCILERS.md`
§13/C1+C4): a
416         bare LOVO mean at n?6 is not trustworthy on its own.
417         """
418         if len(videos) < 1:
419             raise ExperimentError("compare needs at least one labeled video")
420         eval_a = evaluate_variant(videos, variant_a, iou, honest)
421         eval_b = evaluate_variant(videos, variant_b, iou, honest)
422
423         delta = {m: eval_b["aggregate"][m] - eval_a["aggregate"][m] for m in _METRICS}
424         delta_pct = {m: pct_improvement(eval_a["aggregate"][m], eval_b["aggregate"][m]) for
m in _METRICS}
425

```

```

426 by_video_a = {p["video"]: p for p in eval_a["per_video"]}
427 per_video_delta = [
428     {"video": p["video"], **{m: p[m] - by_video_a[p["video"]][m] for m in _METRICS}}
429     for p in eval_b["per_video"]]
430 ]
431
432 # One fingerprint over the whole label set ? detects "the deltas were measured
against
433 # different labels" the same way regression.gt_hash guards the no-regression
baseline.
434 all_gts: List[Interval] = [iv for vc in videos for iv in (vc.gts or [])]
435
436 result: Dict[str, Any] = {
437     "iou": iou,
438     "label_set_hash": gt_hash(all_gts),
439     "num_videos": len(videos),
440     "variant_a": eval_a,
441     "variant_b": eval_b,
442     "delta": delta,
443     "delta_pct": delta_pct,
444     "per_video_delta": per_video_delta,
445 }
446 if honest_stats:
447     result["honest"] = honest_summary(videos, eval_a, eval_b)
448 return result
449
450
451 # ----- SxS on unlabeled video
452
453
454 def compare_unlabeled(video: VideoCandidates, variant_a: Variant, variant_b: Variant,
455     agree_iou: float = 0.5) -> Dict[str, Any]:
456     """Side-by-side on NEW footage with no ground truth (`EXPERIMENT_HARNESS.md` §5.2).
457
458     With no labels there is no lift to measure ? instead this surfaces where the two
459     variants **agree vs diverge**, which is exactly what a human reviews (and what two
460     highlight reels would show side by side):
461
462     - ``agreed`` ? windows both variants found (matched at ``agree_iou``);
463     - ``a_only`` ? A fired, B suppressed (B's added precision, if A was over-firing);
464     - ``b_only`` ? B fired, A did not (B's new detections ? real recall or new FPs:
465       the human / a later golden label adjudicates).
466
467     Both variants must be predictable without labels: a learned variant therefore needs
468     a
469     pinned ``classifier_model`` (you cannot train on an unlabeled video). Reuses
470     ``metrics.match_intervals`` ? no new matching logic.
471     """
472     preds_a = predict(variant_a, video)
473     preds_b = predict(variant_b, video)
474     mr = match_intervals(preds_a, preds_b, agree_iou)
475
476     agreed = [{"a": preds_a[m.pred_index], "b": preds_b[m.gt_index], "iou": m.iou}
477               for m in mr.matches]
478     agree_iou = [m.iou for m in mr.matches]
479     a_only = [preds_a[i] for i in mr.unmatched_pred_indices]
480     b_only = [preds_b[i] for i in mr.unmatched_gt_indices]
481
482     def _dur(ivs: List[Interval]) -> float:
483         return sum(e - s for s, e in ivs)
484
485     return {
486         "video": video.name,
487         "agree_iou": agree_iou,
488         "variant_a": variant_a.name,

```

```

488         "variant_b": variant_b.name,
489         "num_a": len(preds_a),
490         "num_b": len(preds_b),
491         "num_agreed": len(agreed),
492         "mean_agree_iou": (sum(agree_ious) / len(agree_ious)) if agree_ious else 0.0,
493         "duration_a": _dur(preds_a),
494         "duration_b": _dur(preds_b),
495         "agreed": agreed,
496         "a_only": a_only,
497         "b_only": b_only,
498     }
499
500
501 # ----- leaderboard / DB
502
503
504 def record_experiment(result: Dict[str, Any], path: str = DEFAULT_LEADERBOARD_PATH,
505                      timestamp: Optional[str] = None) -> Dict[str, Any]:
506     """Append a compact, reproducible record of a `compare()` result to the JSON
507     leaderboard (`EXPERIMENT_HARNESS.md` §6: experiments are records). Generalises
508     `regression_baseline.json`; deliberately the size of a baseline file, not a service
509     (§9). Returns the row written. ``timestamp`` is injectable for deterministic tests.
510     """
511     row: Dict[str, Any] = {
512         "timestamp": timestamp or datetime.now(timezone.utc).isoformat(),
513         "iou": result.get("iou"),
514         "label_set_hash": result.get("label_set_hash"),
515         "num_videos": result.get("num_videos"),
516         "variant_a": {"name": result["variant_a"]["variant"],
517                      "spec_hash": result["variant_a"]["spec_hash"],
518                      "aggregate": result["variant_a"]["aggregate"]},
519         "variant_b": {"name": result["variant_b"]["variant"],
520                      "spec_hash": result["variant_b"]["spec_hash"],
521                      "aggregate": result["variant_b"]["aggregate"]},
522         "delta": result.get("delta"),
523     }
524     # Record the honest ?F1 (CI + sign test) when present, so the leaderboard carries
525     the
526     # promotion-grade evidence, not just the bare-mean delta (additive ? C1).
527     honest = result.get("honest") or {}
528     if honest.get("paired_delta_f1"):
529         row["honest_delta_f1"] = honest["paired_delta_f1"]
530     entries: List[Dict[str, Any]] = []
531     if os.path.isfile(path):
532         with open(path, encoding="utf-8") as f:
533             entries = json.load(f)
534     entries.append(row)
535     os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
536     with open(path, "w", encoding="utf-8") as f:
537         json.dump(entries, f, indent=2)
538     logger.info("recorded experiment %s vs %s (?f1=%.3f) -> %s",
539               row["variant_a"]["name"], row["variant_b"]["name"],
540               (result.get("delta") or {}).get("f1", 0.0), path)
541     return row
542
543 def load_video_candidates(db: Any, video_id: str, *, detector: Optional[str] = None,
544                          gts: Optional[List[Interval]] = None,
545                          feature_names: Sequence[str] = ()) -> VideoCandidates:
546     """Bridge persisted detection ? `VideoCandidates` via
547     `Database.query_candidate_segments`.
548     ``feature_names`` selects which persisted features to lift out of each row's
549     ``stats``
550     JSON (``"duration"`` is derived from the window bounds). This is the only DB-

```

```

touching
550         function; the comparison itself is pure and offline. Exercised owner-side on a real
DB,
551         but works against any `Database` (including an in-memory one in tests).
552         ""
553         rows = db.query_candidate_segments(video_id, detector=detector)
554         intervals: List[Interval] = [(float(r["start_time"]), float(r["end_time"])) for r in
rows]
555         features: Dict[str, List[float]] = {}
556         for name in feature_names:
557             if name == "duration":
558                 features[name] = [float(r["end_time"]) - float(r["start_time"]) for r in
rows]
559             else:
560                 features[name] = [float((r.get("stats") or {}).get(name, 0.0)) for r in
rows]
561         return VideoCandidates(name=video_id, intervals=intervals, features=features,
gts=gts)
562

```

23. assistant (2026-06-14T10:37:13.586Z)

Now I have a full picture. Let me clarify the critical architectural distinction that drives the test design.

****Offline ablation litmus**** (the +0.074 source): builds candidates via ``distill_local.build_candidates`` with ``PROPOSAL = (0.005, 1.0, 0.5)`` (recall-oriented, permissive windowing), then the experiment harness ``_gate_mask`` applies ``min_crossings`` over the persisted ``net_crossings`` feature. The over-seg ratio 1.68?1.01 and ?F1 +0.074 come from this.

****Served path**** (``FusionSegmenter``): builds candidates via ``trajectory_to_action_windows`` with PRODUCTION config (``velocity_thresh=0.02, merge_gap=2.0, min_window_duration=2.0`` ? the ``BASELINE_LOCAL``, not the permissive proposal), computes ``net_crossings`` via ``_enrich_candidate_stats`` (calling ``rally_gate.gate_windows(..., min_crossings=0)``), then ``_select_for_handoff`` drops windows with ``net_crossings < min_crossings``.

The served path's candidate windowing differs from the offline harness's permissive proposal. So the served numbers may diverge from the offline +0.074 ? and the task explicitly asks me to report that divergence honestly. The served path is what production actually runs.

Let me check the production default config to confirm ``min_crossings=2`` and ``default_segmenter=fusion_hybrid``, and look at ``trajectory_to_action_windows`` and ``WasbConfig`/`TrackNetConfig``.

24. user (2026-06-14T10:37:15.631Z)

```

26:         "default_segmenter": "fusion_hybrid",
30:         "dead_time_merge_gap": 2.0,
35:         "yolo_velocity_threshold": 0.15,
39:         "min_action_window_duration": 2.0,
54:         "velocity_threshold": 0.02
65:         "velocity_threshold": 0.02
67:         "fusion": {
69:             "min_crossings": 2,

```

25. user (2026-06-14T10:37:16.758Z)

```

233:     def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
252:     def trajectory_to_action_windows(

```

26. assistant (2026-06-14T10:37:26.953Z)

Let me read the config fusion block fully and the ``trajectory_to_action_windows`` signature/body.

27. user (2026-06-14T10:37:28.007Z)

```
20         255
21     ],
22     "court_pixels_min_ratio": 0.05
23 }
24 },
25 "indexing": {
26     "default_segmenter": "fusion_hybrid",
27     "use_gpu": true,
28     "motion_threshold": 0.08,
29     "min_segment_duration": 3.0,
30     "dead_time_merge_gap": 2.0,
31     "chunk_duration": 90,
32     "chunk_overlap": 10,
33     "detector_model": "fasterrcnn_resnet50_fpn",
34     "detector_score_threshold": 0.5,
35     "yolo_velocity_threshold": 0.15,
36     "yolo_frame_skip": 3,
37     "yolo_tracker": "bytetrack.yaml",
38     "yolo_prefetch_workers": 2,
39     "min_action_window_duration": 2.0,
40     "log_yolo_candidates": true,
41     "store_yolo_candidates": true,
42     "ai_handoff_padding": 2.0,
43     "ai_max_workers": 1,
44     "tracknet": {
45         "wsl_distro": "Ubuntu",
46         "repo_dir": "~/models/TrackNetV4",
47         "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
48         "conda_env": "TrackNetV4",
49         "weights_path": "",
50         "queue_length": 5,
51         "stage_in_wsl": true,
52         "wsl_stage_dir": "~/clips",
53         "timeout_sec": 1800,
54         "velocity_threshold": 0.02
55     },
56     "wasb": {
57         "wsl_distro": "Ubuntu",
58         "repo_dir": "~/models/WASB-SBDT",
59         "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
60         "conda_env": "wasb",
61         "weights_path": "~/models/WASB-
62         SBDT/pretrained_weights/wasb_badminton_best.pth.tar",
63         "sport": "badminton",
64         "wsl_stage_dir": "~/clips",
65         "timeout_sec": 1800,
66         "velocity_threshold": 0.02
67     },
68     "fusion": {
69         "substrate": "wasb",
70         "min_crossings": 2,
71         "min_players": 0,
72         "cue_flags": {}
73     },
74     "output": {
75         "default_highlights_name": "highlights.mp4",
76         "db_name": "sports_indexer.db"
77     },
78     "stitching": {
79         "begin_padding": 0.5,
```

28. user (2026-06-14T10:37:28.450Z)

```

230 # ----- CSV -> windows
231
232 @staticmethod
233 def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
234     """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
235     points: List[TrajectoryPoint] = []
236     with open(csv_win_path, newline="") as f:
237         reader = csv.DictReader(f)
238         for row in reader:
239             try:
240                 points.append(TrajectoryPoint(
241                     frame=int(row["Frame"]),
242                     visible=int(row["Visibility"]) == 1,
243                     x=float(row["X"]),
244                     y=float(row["Y"]),
245                 ))
246             except (KeyError, ValueError):
247                 continue
248         points.sort(key=lambda p: p.frame)
249     return points
250
251 @staticmethod
252 def trajectory_to_action_windows(
253     points: List[TrajectoryPoint],
254     fps: float,
255     frame_width: float,
256     chunk_start: float = 0.0,
257     velocity_thresh: float = 0.02,
258     merge_gap: float = 2.0,
259     min_window_duration: float = 2.0,
260     chunk_index: Optional[int] = None,
261 ) -> List[Dict[str, Any]]:
262     """Convert a shuttle trajectory into candidate rally windows.
263
264     A frame is "active" when the shuttle is visible AND its inter-frame
265     displacement (normalised by frame width, per second) exceeds
266     ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
267     ``merge_gap`` seconds of each other are grouped into one window; short
268     windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
269     shape feeding the AI-handoff phase is identical.
270     """
271     if fps <= 0:
272         fps = 30.0
273     if frame_width <= 0:
274         frame_width = 1280.0
275
276     # Compute per-frame normalised velocity from the last *visible* point.
277     active = [] # (frame_idx, velocity)
278     prev: Optional[TrajectoryPoint] = None
279     for p in points:
280         if not p.visible:
281             prev = None # break the trajectory; absent shuttle = not in flight
282             continue
283         if prev is None:
284             dframes = p.frame - prev.frame
285             if dframes > 0:
286                 dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
287                 velocity = (dist / frame_width) * (fps / dframes)
288                 if velocity > velocity_thresh:
289                     active.append((p.frame, velocity))
290             prev = p
291
292     if not active:
293         return []
294

```

```

295         max_gap_frames = int(merge_gap * fps)
296
297     def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
298         vels = [v for _, v in group]
299         return {
300             "start": group[0][0] / fps + chunk_start,
301             "end": group[-1][0] / fps + chunk_start,
302             "active_frame_count": len(group),
303             "peak_velocity": max(vels),
304             "mean_velocity": sum(vels) / len(vels),
305             "track_id_count": 1, # single shuttle; kept for window-shape parity
306             "chunk_index": chunk_index,
307         }
308
309     windows = []
310     group = [active[0]]
311     for af in active[1:]:
312         if af[0] - group[-1][0] <= max_gap_frames:
313             group.append(af)
314         else:
315             windows.append(_finalize(group))
316             group = [af]
317     windows.append(_finalize(group))
318
319     return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
320

```

29. assistant (2026-06-14T10:37:36.580Z)

I now have everything I need. The design is clear. Let me write a runner helper that exercises the `real production `FusionSegmenter`` selection logic without GPU/Gemini.

The cleanest, most honest approach: reuse the actual `FusionSegmenter._enrich_candidate_stats`` and `_select_for_handoff`` methods (the served decision logic) on windows produced by the production windowing path. I'll instantiate a real `FusionSegmenter`` (with a dummy DB, since I only call the seam methods, not `process_video``), feed it cached-trajectory windows built by `trajectory_to_action_windows`` with the production config, and let its own methods compute `net_crossings`` and select.

Let me verify the golden trajectory/label files exist and check `IndexingConfig`/load_config`` so the runner uses the real production config defaults.

30. user (2026-06-14T10:37:39.840Z)

```

total 9416
drwxr-xr-x 1 avidu 197609      0 Jun 10 12:56 .
drwxr-xr-x 1 avidu 197609      0 Jun 14 07:33 ..
-rw-r--r-- 1 avidu 197609 2476375 Jun  8 18:24 adarsh_avi_full_traj.csv
-rw-r--r-- 1 avidu 197609 1114525 Jun  8 14:24 adarsh_avi_traj.csv
-rw-r--r-- 1 avidu 197609 1101363 Jun  6 20:53 adarsh_traj.csv
-rw-r--r-- 1 avidu 197609 1702622 Jun  8 19:11 gbaaddy_traj.csv
-rw-r--r-- 1 avidu 197609 1159102 Jun  8 18:42 kushagra_traj.csv
-rw-r--r-- 1 avidu 197609 693876 Jun  8 21:02 largetestvideo_traj.csv
-rw-r--r-- 1 avidu 197609 418680 Jun  9 10:40 mahadevpura_traj.csv
-rw-r--r-- 1 avidu 197609 145287 Jun  9 18:10 pickleball_traj.csv
drwxr-xr-x 1 avidu 197609      0 Jun 10 12:56 roora
-rw-r--r-- 1 avidu 197609 600480 Jun  6 18:01 sample_long_traj.csv
-rw-r--r-- 1 avidu 197609 96822 Jun  9 18:13 tabletennis_traj.csv
-rw-r--r-- 1 avidu 197609 96359 Jun  6 22:20 testlarge_traj.csv
---LABELS---
total 38883731
drwxr-xr-x 1 avidu 197609      0 Jun 10 17:38 .
drwxr-xr-x 1 avidu 197609      0 Jun 14 07:33 ..
-rw-r--r-- 1 avidu 197609 11445052772 Jun  1 08:38 Adarsh and Avi.MP4
-rw-r--r-- 1 avidu 197609      4106 Jun  8 17:20 Adarsh and Avi.rallies.csv

```



```

-rw-r--r-- 1 avidu 197609 8058214679 May 22 08:02 KushagraYashAviFirstVideo.MP4
-rw-r--r-- 1 avidu 197609 1413 Jun 8 17:41 KushagraYashAviFirstVideo.rallies.csv
-rw-r--r-- 1 avidu 197609 3271 Jun 10 17:38 KushagraYashAviFirstVideo.report.json
-rw-r--r-- 1 avidu 197609 1611 Jun 10 17:38 KushagraYashAviFirstVideo.report.md
-rw-r--r-- 1 avidu 197609 493 Jun 10 17:38 KushagraYashAviFirstVideo.summary.md
-rw-r--r-- 1 avidu 197609 11531882761 May 4 17:18 LargeTestVideo.MP4
-rw-r--r-- 1 avidu 197609 2140 Jun 8 20:28 LargeTestVideo.rallies.csv
-rw-r--r-- 1 avidu 197609 196182380 Jan 16 08:00 MahadevpuraSingles.MP4
-rw-r--r-- 1 avidu 197609 1325 Jun 9 08:08 MahadevpuraSingles.rallies.csv
-rw-r--r-- 1 avidu 197609 3222 Jun 10 16:29 MahadevpuraSingles.report.json
-rw-r--r-- 1 avidu 197609 1571 Jun 10 16:29 MahadevpuraSingles.report.md
-rw-r--r-- 1 avidu 197609 486 Jun 10 16:29 MahadevpuraSingles.summary.md
-rw-r--r-- 1 avidu 197609 1510427943 May 4 20:39 TestLargeVideo.MP4
-rw-r--r-- 1 avidu 197609 287 Jun 8 21:33 TestLargeVideo.rallies.csv
-rw-r--r-- 1 avidu 197609 7075098663 May 22 08:40 gBaaddy20May2026.MP4
---GOLDEN FEATURES---
C:/Users/avidu/Projects/badminton-highlight-indexer/output/adarsh_avi_singles_golden_features.json
C:/Users/avidu/Projects/badminton-highlight-indexer/output/gbaaddy_golden_features.json
C:/Users/avidu/Projects/badminton-highlight-indexer/output/kushagra_singles_golden_features.json
C:/Users/avidu/Projects/badminton-highlight-indexer/output/largetest_doubles_golden_features.json
C:/Users/avidu/Projects/badminton-highlight-indexer/output/mahadevpura_singles_golden_features.json
C:/Users/avidu/Projects/badminton-highlight-indexer/output/testlarge_short_golden_features.json

```

31. user (2026-06-14T10:37:40.912Z)

```

backend\config\models.py:78:class FusionConfig(BaseModel):
backend\config\models.py:81:    **Gate A (the held-shuttle guard) is ON by default**
(``min_crossings=2``): a window
backend\config\models.py:86:    is ``fusion_hybrid`` (config.json
``indexing.default_segmenter``); the offline experiment
backend\config\models.py:87:    harness is unaffected (it sets ``Variant.min_crossings``
explicitly, never reading this).
backend\config\models.py:89:    **Reversible ? rollback is a single flag:** set
``indexing.fusion.min_crossings: 0`` to
backend\config\models.py:90:    restore the pre-Gate-A (ungated) handoff, or
``indexing.default_segmenter: wasb_hybrid``
backend\config\models.py:91:    to revert the served segmenter entirely. With
``min_crossings=0`` the fusion segmenter
backend\config\models.py:116:    min_crossings: int = Field(default=2, ge=0)
backend\config\models.py:128:class IndexingConfig(BaseModel):
backend\config\models.py:129:    default_segmenter: str = "yolo_hybrid"
backend\config\models.py:161:    @field_validator("default_segmenter")
backend\config\models.py:240:class AppConfig(BaseModel):
backend\config\loader.py:53:def load_config(path: str | Path | None = None) -> AppConfig:

```

32. assistant (2026-06-14T10:37:44.821Z)

All 6 golden trajectories and labels are present, and the offline golden feature JSONs exist (so the existing litmus runs on this box). Let me check the WSL environment to understand whether torch/cv2 are importable here (the served path imports cv2 at module level in `trajectory\_hybrid.py`). This matters because importing `FusionSegmenter` pulls in `trajectory\_hybrid` ? `cv2`. Let me check.

33. user (2026-06-14T10:37:53.395Z)

```

cv2 OK 4.13.0
---numpy---
numpy OK
---import FusionSegmenter---
FusionSegmenter import OK

```

34. assistant (2026-06-14T10:37:59.411Z)

cv2, numpy, and `FusionSegmenter` all import cleanly on this box. This box is the "owner box" for this purpose (golden data + cv2 present). Now let me build the runner helper and run the served-path verification.

First, let me write the runner helper. I'll create `backend/eval/served\_gate\_a.py` ? a small helper that runs the **production** `FusionSegmenter` seam methods** (`\_enrich\_candidate\_stats` + `\_select\_for\_handoff`) on a cached trajectory, producing the kept handoff windows for a given config, then scores them vs golden GTs.

35. user (2026-06-14T10:38:00.786Z)

```
78     class FusionConfig(BaseModel):
79         """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN
P2).
80
81         **Gate A (the held-shuttle guard) is ON by default** (`min_crossings=2`): a window
82         must show the shuttle crossing the net >= 2 times to reach the AI handoff, dropping
the
83         held-shuttle / one-sided-feed / warm-up false positives. This is the promoted,
validated
84         operating point (?F1 +0.074, 6/6 golden, sign-test p=0.031, CI [+0.042,+0.105];
over-seg
85         1.68?1.01 ? see docs/QUALITY_ITERATIONS.md). It only takes effect when the served
segmenter
86         is `fusion_hybrid` (config.json `indexing.default_segmenter`); the offline
experiment
87         harness is unaffected (it sets `Variant.min_crossings` explicitly, never reading
this).
88
89         **Reversible ? rollback is a single flag:** set `indexing.fusion.min_crossings: 0`
to
90         restore the pre-Gate-A (ungated) handoff, or `indexing.default_segmenter:
wasb_hybrid`
91         to revert the served segmenter entirely. With `min_crossings=0` the fusion
segmenter
92         reproduces the substrate (wasb_hybrid) byte-for-byte.
93
94         The person cue (Gate B) stays OFF by default: the fusion segmenter persists the
shuttle
95         net-crossing count always, and ? only when `cue_flags['person']` is true ? the
person-cue
96         features; `min_players` (Gate B) defaults to 0 = OFF. The court mask is optional:
97         `court_polygon=None` ? Gate B counts every detection (player-count-only); supplied
?
98         off-court persons (spectators / adjacent courts) are excluded. **The persisted
candidate
99         set is the full superset regardless of any gate** ? gating only filters the AI
handoff, so
100         the offline harness can re-score any variant.
101         """
102         # Which trajectory substrate seeds the windows (shuttle stays primary).
103         substrate: Literal["wasb", "tracknet"] = "wasb"
104         # Windowing velocity threshold for the fusion family (the engine reads
105         # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
106         # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
107         velocity_threshold: float = 0.02
108         # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
109         # enabled ? so the default path never imports torch / touches the GPU.
110         cue_flags: dict[str, bool] = Field(default_factory=dict)
111         # Served Gate A: drop windows whose shuttle net-crossings < this from the AI
handoff.
112         # Default 2 = the PROMOTED held-shuttle guard (a real rally crosses the net >=2x; a
single
```

```

113         # service feed / held shuttle crosses ~once). Set 0 to turn the gate OFF (ungated
handoff;
114         # reproduces the substrate exactly). Persisted candidates are ALWAYS the full
superset for
115         # offline re-scoring, regardless of this threshold.
116         min_crossings: int = Field(default=2, ge=0)
117         # Served Gate B: when the person cue is on, drop windows with median in-court
players
118         # < this. 0 = OFF.
119         min_players: int = Field(default=0, ge=0)
120         # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
121         court_polygon: Optional[list[list[float]]] = None
122         # Net line for the person two-sided-motion split (person features only ? the shuttle
123         # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
124         net_axis: Literal["x", "y"] = "y"
125         net_position: float = Field(default=0.5, ge=0.0, le=1.0)
126
127
128     class IndexingConfig(BaseModel):
129         default_segmenter: str = "yolo_hybrid"
130         use_gpu: bool = True
131         motion_threshold: float = 0.08
132         min_segment_duration: float = 3.0

```

36. user (2026-06-14T10:38:01.353Z)

```

1     from abc import ABC, abstractmethod
2     from typing import List, Dict, Any, Type, Optional
3     from backend.infrastructure.database import Database
4     from backend.results import Result # new standardized contract (Result[Value] ==
Success[Value] | Failure)
5
6     class BasePlaySegmenter(ABC):
7         def __init__(self, db: Database, config: Dict[str, Any]):
8             self.db = db
9             self.config = config
10
11         @property
12         @abstractmethod
13         def name(self) -> str:
14             """Unique identifier for this segmenter."""
15             pass
16
17         @property
18         @abstractmethod
19         def description(self) -> str:
20             """Human-readable explanation of this segmenter."""
21             pass
22
23         @abstractmethod
24         def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
25             """
26             Processes a video to detect play segments, populates SQLite database,
27             and returns a Result.
28
29             Use isinstance(result, Failure) (or result.is_success()) to distinguish
30             "no segments found" from "something went wrong" (missing key, healthcheck
31             failure, etc.). This is the start of the standardized error/result contract.
32             """
33             pass
34
35     class SegmenterRegistry:
36         def __init__(self):
37             self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}

```

```

38
39     def register(self, segmenter_cls: Type[BasePlaySegmenter]):
40         """Registers a new segmenter class under its name property."""
41         try:
42             temp_inst = segmenter_cls(None, {})
43             name = temp_inst.name
44             except Exception as e:
45                 raise ValueError(f"Failed to resolve name property from segmenter class
{segmenter_cls.__name__}: {e}") from e
46             self._segmenters[name] = segmenter_cls
47
48     def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
49         return self._segmenters.get(name)
50
51     def list_available(self) -> Dict[str, str]:
52         """Returns a dict of registered segmenter names and their descriptions."""
53         res = {}
54         for name, cls in self._segmenters.items():
55             try:
56                 temp_inst = cls(None, {})
57                 desc = temp_inst.description
58             except Exception:
59                 desc = "No description available."
60             res[name] = desc

```

37. assistant (2026-06-14T10:38:06.821Z)

I have everything needed. `FusionSegmenter(None, {})` is constructible (the registry does exactly this). Now I'll write the runner helper. The key design decision: I'll instantiate a `real FusionSegmenter` with a `config` containing the production indexing config, and call its `actual production seam methods` ? `_enrich_candidate_stats` (computes `net_crossings`) and `_select_for_handoff` (the gate) ? on windows built by the production windowing path. This exercises the real served code, not a reimplementation.

Let me write the runner helper.

38. assistant (2026-06-14T10:38:44.810Z)

""SERVED Gate-A verification: run the `production` `FusionSegmenter` decision seams on a cached WASB trajectory and score the resulting AI-handoff windows vs golden labels.

The offline litmus (`backend.eval.ablation` / `fusion_compare`) measures Gate A on the `permissive proposal windowing` (`distill_local.PROPOSAL` = recall-oriented) re-scored through the experiment harness's `_gate_mask`. That validates the `signal`. It does NOT exercise the `served path`: in production the windows come from `trajectory_to_action_windows` at the production operating point (`velocity_threshold=0.02, dead_time_merge_gap=2.0, min_action_window_duration=2.0`), the net-crossing count is computed by `FusionSegmenter._enrich_candidate_stats`, and the gate is `FusionSegmenter._select_for_handoff`. THIS module drives exactly those production methods ? no reimplementation of the windowing or the gate ? on a cached trajectory CSV, so the verification reflects what `process_video` actually hands to Gemini.

GPU-free and Gemini-free: it stops at the handoff-selection boundary (the windows that WOULD be sent to the AI). With the person cue OFF (the default) nothing here loads torch or touches the GPU ? it just re-scores cached WASB trajectories.

Run (owner box, golden trajectories present):

```
python -m backend.eval.served_gate_a --manifest output/human_lovo_manifest.json
"""
```

```
from __future__ import annotations
```

```
import argparse
```

```
import json
```

```
from dataclasses import dataclass
```

```
from typing import Any, Dict, List, Optional, Tuple
```

```

from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
from backend.eval.calibrate_wasb import load_golden_gts
from backend.eval.metrics import Interval, score, Score
from backend.eval import segmentation_metrics as _segm

```

Production windowing operating point (config.json indexing defaults ? the values TrajectoryHybridSegmenter.process_video reads for the served fusion run).

```

PROD_VELOCITY_THRESH = 0.02
PROD_MERGE_GAP = 2.0
PROD_MIN_WINDOW = 2.0

```

```

def _served_idx_cfg(min_crossings: int) -> Dict[str, Any]:
    """The ``indexing`` config dict the served FusionSegmenter seams read ? mirrors
    config.json's ``indexing.fusion`` block, with Gate A at ``min_crossings`` and the
    person cue OFF (so no GPU / torch is touched)."""
    return {
        "fusion": {
            "substrate": "wasb",
            "min_crossings": min_crossings,
            "min_players": 0,
            "cue_flags": {},
        },
        "wasb": {"velocity_threshold": PROD_VELOCITY_THRESH},
        "dead_time_merge_gap": PROD_MERGE_GAP,
        "min_action_window_duration": PROD_MIN_WINDOW,
    }

def served_handoff_windows(trajecory_csv: str, fps: float, frame_width: float,
                           min_crossings: int) -> List[Interval]:
    """Run the PRODUCTION FusionSegmenter decision path on a cached trajectory and return the
    windows that would reach the AI handoff (i.e. the served rally candidates).

    Uses the real ``FusionSegmenter`` seam methods ? ``_enrich_candidate_stats`` (computes
    ``net_crossings``) and ``_select_for_handoff`` (Gate A) ? over windows from the production
    ``trajectory_to_action_windows`` operating point. No GPU, no Gemini: the person cue is OFF.
    """
    points = TrackNetRunner.parse_trajectory_csv(trajecory_csv)
    windows = TrackNetRunner.trajectory_to_action_windows(
        points, fps=fps, frame_width=frame_width,
        velocity_thresh=PROD_VELOCITY_THRESH, merge_gap=PROD_MERGE_GAP,
        min_window_duration=PROD_MIN_WINDOW,
    )
    idx_cfg = _served_idx_cfg(min_crossings)
    # Construct the real served segmenter (db=None, config carries the indexing block). We only
    # call the pure decision seams ? no process_video, no DB, no provider ? exactly as the
    # registry constructs it for `name`/`description`.
    seg = FusionSegmenter(None, {"indexing": idx_cfg, "sport": "badminton"})
    # The served per-window stats (incl. net_crossings) via the production enrichment hook.
    window_stats = [
        seg._enrich_candidate_stats(cw, points, fps, frame_width, video_path="",
                                   idx_cfg=idx_cfg)
        for cw in windows
    ]
    keep = seg._select_for_handoff(windows, window_stats, idx_cfg)
    return [(windows[i]["start"], windows[i]["end"]) for i in keep]

```

```

@dataclass
class ServedVideoResult:
    name: str

```

```

num_gt: int
on: Score          # Gate-A ON (min_crossings=2)
off: Score          # Gate-A OFF (min_crossings=0, control)
seg_ratio_on: float # |preds| / |gts| with Gate-A ON (over-segmentation)
seg_ratio_off: float
n_handoff_on: int
n_handoff_off: int

@property
def delta_f1(self) -> float:
    return self.on.f1 - self.off.f1

def evaluate_video(spec: Dict[str, Any], iou: float = 0.5,
                  on_crossings: int = 2) -> ServedVideoResult:
    """Score the served Gate-A ON (min_crossings=on_crossings) vs OFF (0) handoff windows for
    one golden video against its golden labels."""
    fps, fw = float(spec["fps"]), float(spec["frame_width"])
    traj, labels = str(spec["trajectory"]), str(spec["labels"])
    gts = load_golden_gts(labels)
    preds_on = served_handoff_windows(traj, fps, fw, min_crossings=on_crossings)
    preds_off = served_handoff_windows(traj, fps, fw, min_crossings=0)
    return ServedVideoResult(
        name=str(spec["name"]),
        num_gt=len(gts),
        on=score(preds_on, gts, iou),
        off=score(preds_off, gts, iou),
        seg_ratio_on=_segm.segment_count_ratio(preds_on, gts),
        seg_ratio_off=_segm.segment_count_ratio(preds_off, gts),
        n_handoff_on=len(preds_on),
        n_handoff_off=len(preds_off),
    )

def evaluate_manifest(specs: List[Dict[str, Any]], iou: float = 0.5,
                    on_crossings: int = 2) -> List[ServedVideoResult]:
    return [evaluate_video(s, iou, on_crossings) for s in specs]

def aggregate(results: List[ServedVideoResult]) -> Dict[str, Any]:
    """Mean ON/OFF F1 + over-seg ratio, the paired ?F1, and the per-video win count."""
    n = len(results)

    def _mean(getter) -> float:
        return sum(getter(r) for r in results) / n if n else 0.0

    deltas = [r.delta_f1 for r in results]
    wins = sum(1 for d in deltas if d > 1e-9)
    losses = sum(1 for d in deltas if d < -1e-9)
    return {
        "num_videos": n,
        "mean_f1_on": _mean(lambda r: r.on.f1),
        "mean_f1_off": _mean(lambda r: r.off.f1),
        "mean_delta_f1": _mean(lambda r: r.delta_f1),
        "mean_precision_on": _mean(lambda r: r.on.precision),
        "mean_precision_off": _mean(lambda r: r.off.precision),
        "mean_recall_on": _mean(lambda r: r.on.recall),
        "mean_recall_off": _mean(lambda r: r.off.recall),
        "mean_seg_ratio_on": _mean(lambda r: r.seg_ratio_on),
        "mean_seg_ratio_off": _mean(lambda r: r.seg_ratio_off),
        "wins": wins,
        "losses": losses,
    }

```

```

def format_table(results: List[ServedVideoResult], agg: Dict[str, Any], iou: float) -> str:
    lines = [
        f"=== SERVED Gate-A verification (production FusionSegmenter path, IoU>={iou}) ===",
        f" {'video':<24s} {'gt':>3s} {'F1_off':>7s} {'F1_on':>7s} {'?F1':>7s} "
        f"{'segR_off':>9s} {'segR_on':>8s} {'hand_off':>9s} {'hand_on':>8s}",
    ]
    for r in results:
        lines.append(
            f" {r.name:<24s} {r.num_gt:>3d} {r.off.f1:>7.3f} {r.on.f1:>7.3f} "
            f"{r.delta_f1:>+7.3f} {r.seg_ratio_off:>9.2f} {r.seg_ratio_on:>8.2f} "
            f"{r.n_handoff_off:>9d} {r.n_handoff_on:>8d}")
    lines += [
        f" {'-' * 92}",
        f" MEAN F1 {agg['mean_f1_off']:.3f} (off) -> {agg['mean_f1_on']:.3f} (on) "
        f"?F1={agg['mean_delta_f1']:+.3f} "
        f"P {agg['mean_precision_off']:.3f}->{agg['mean_precision_on']:.3f} "
        f"R {agg['mean_recall_off']:.3f}->{agg['mean_recall_on']:.3f}",
        f" over-seg ratio {agg['mean_seg_ratio_off']:.2f} (off) -> "
        f"{agg['mean_seg_ratio_on']:.2f} (on) | wins {agg['wins']}/{agg['num_videos']} "
        f"(losses {agg['losses']}]",
    ]
    return "\n".join(lines)

def run(manifest_path: str, iou: float = 0.5) -> Tuple[List[ServedVideoResult], Dict[str, Any]]:
    with open(manifest_path, encoding="utf-8") as f:
        specs = json.load(f)
    results = evaluate_manifest(specs, iou=iou)
    return results, aggregate(results)

def main(argv: Optional[List[str]] = None) -> None:
    ap = argparse.ArgumentParser(
        description="Verify served Gate-A on real footage via the production FusionSegmenter path.")
    ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
    ap.add_argument("--iou", type=float, default=0.5)
    args = ap.parse_args(argv)
    results, agg = run(args.manifest, iou=args.iou)
    print(format_table(results, agg, args.iou))

if __name__ == "__main__":
    main()

```

39. user (2026-06-14T10:38:44.827Z)

File created successfully at: C:\Users\avidu\Projects\bhi-servedverify\backend\eval\served_gate_a.py (file state is current in your context ? no need to Read it back)

40. assistant (2026-06-14T10:38:48.259Z)

Let me check `segmentation\_metrics.segment\_count\_ratio` signature to confirm I'm calling it correctly, then run the verification against the main checkout's manifest.

41. user (2026-06-14T10:38:49.910Z)

```

110: def segment_count_ratio(preds: List[Interval], gts: List[Interval]) -> float:
111-     """Predicted-to-GT segment-count ratio ? a crude fragmentation diagnostic.
112-
113-     ``> 1`` hints over-segmentation (more predicted pieces than rallies), ``< 1`` hints
114-     merging/misses. Only meaningful with ground truth present; returns ``0.0`` if ``gts``
115-     is empty (use F1@k / mAP for the graded picture).

```

```

116-     ""
117-     if not gts:
118-         return 0.0
119-     return len(preds) / len(gts)
120-
121-
122-def segmentation_report(scored_preds: Sequence[ScoreInterval], gts: List[Interval],
123-                          overlaps: Sequence[float] = DEFAULT_OVERLAPS,
124-                          iou_thresholds: Sequence[float] = DEFAULT_TIOUS) -> Dict[str,
125-                          object]:
126-     """One-call bundle: F1@k + mAP@tIoU + per-tIoU AP + count ratio.
127-     ``scored_preds`` carry confidences for AP; F1@k uses the bare intervals. Returns a
128-     plain dict (JSON-friendly, string keys) so it can ride next to a ``compare`` result

```

42. assistant (2026-06-14T10:38:52.770Z)

The signature matches. Now let me run the served-path verification against the main checkout's manifest.

43. user (2026-06-14T10:39:02.538Z)

```

=== SERVED Gate-A verification (production FusionSegmenter path, IoU>=0.5) ===
video      gt  F1_off  F1_on   ?F1  segR_off  segR_on  hand_off  hand_on
adarsh_avi_singles    96   0.632   0.628  -0.004    1.01    0.79      97      76
kushagra_singles      32   0.000   0.000  +0.000    0.06    0.06       2       2
largetest_doubles     49   0.535   0.484  -0.051    1.06    0.86     52     42
gbaaddy             48   0.086   0.088  +0.003    0.46    0.42     22     20
testlarge_short       6   0.182   0.182  +0.000    0.83    0.83       5       5
mahadevpura_singles  31   0.364   0.370  +0.007    0.77    0.74     24     23
-----
MEAN  F1 0.300 (off) -> 0.292 (on)  ?F1=-0.008  P 0.317->0.337  R 0.290->0.261
over-seg ratio 0.70 (off) -> 0.62 (on)  | wins 2/6 (losses 2)

```

44. assistant (2026-06-14T10:39:10.814Z)

This is a very important finding. The served path with **production windowing** does NOT reproduce the offline +0.074. In fact ?F1 is slightly **negative** (?0.008), with over-seg ratio already < 1 (under-segmentation, not over).

The reason is now clear and is exactly the divergence the task warned me to report honestly: the offline litmus uses the **permissive proposal windowing** (`PROPOSAL = (0.005, 1.0, 0.5)` ? low velocity threshold, short merge gap, short min duration ? MANY small candidate windows, over-seg 1.68), where Gate A cuts the junk. But the **production windowing** (`0.02, 2.0, 2.0`) is already much more conservative ? it produces FEWER, longer windows (over-seg ~0.70, i.e. under one prediction per GT rally), so Gate A has little junk left to cut and even trims a few true positives.

This is a real, material divergence between the offline ablation and the served default. Let me verify this is genuinely the windowing difference (not a bug in my runner) by reproducing the offline +0.074 with the **permissive** proposal windowing through the same served seams, and cross-check against the persisted golden feature JSONs.

45. user (2026-06-14T10:39:16.969Z)

```

num videos: 6
OFFLINE litmus: signal=gate_a ?F1=+0.0739 wins=6/6 p=0.0312 segR_full(on)=1.014
segR_ablated(off)=1.675

```

46. assistant (2026-06-14T10:39:21.178Z)

The offline litmus reproduces exactly (+0.0739, 6/6, p=0.031, over-seg 1.675?1.014). So my environment is correct ? the divergence is real and attributable to windowing. Let me now confirm the root cause directly: run the served seams on the **permissive proposal windowing**

(the offline candidate set) and check whether Gate A then reproduces the offline win. This isolates "windowing" as the sole cause.

47. user (2026-06-14T10:39:33.282Z)

```
adarsh_avi_singles      F1 0.597->0.722 dF1=+0.125 segR 1.79->0.88 n_off=172 n_on=84
kushagra_singles       F1 0.154->0.163 dF1=+0.009 segR 0.62->0.53 n_off=20 n_on=17
largetest_doubles      F1 0.593->0.667 dF1=+0.074 segR 1.76->0.78 n_off=86 n_on=38
gbaaddy               F1 0.231->0.274 dF1=+0.043 segR 1.71->0.98 n_off=82 n_on=47
testlarge_short        F1 0.300->0.375 dF1=+0.075 segR 2.33->1.67 n_off=14 n_on=10
mahadevpura_singles    F1 0.455->0.571 dF1=+0.117 segR 1.84->1.26 n_off=57 n_on=39
MEAN proposal-windowing: F1 0.388->0.462 dF1=+0.0739 segR 1.68->1.01 wins=6/6
```

48. assistant (2026-06-14T10:39:45.892Z)

****This is definitive.**** The root cause is conclusively isolated:

- ****Permissive proposal windowing**** (offline harness candidate set) + Gate A ? ****+0.0739 ?F1, 6/6 wins, over-seg 1.68?1.01**** ? exactly reproduces the offline litmus (the simple-merge raw-gate version even matches it to 4 decimals; the LogisticRegression-based ablation gives the same headline).

- ****Production windowing**** (``0.02, 2.0, 2.0``) + the same Gate A ? ****?0.008 ?F1, over-seg already 0.70****.

The divergence is ****entirely the windowing operating point****, not a bug. Production's conservative windowing (high velocity threshold, 2s merge gap, 2s min duration) merges flight bursts into few long windows and drops short ones, so it produces ***fewer than one*** prediction per GT rally (under-segmentation, segR 0.70) ? there is almost no held-shuttle junk left for Gate A to remove, and it occasionally trims a borderline true window.

This is a genuine, material finding: ****the served default (``fusion_hybrid`` + ``min_crossings=2``) does NOT reproduce the offline +0.074 on the production windowing ? Gate A is roughly neutral-to-slightly-negative there.**** Per the task, I must NOT merge a misleading "pass." Let me capture the production windowing's candidate counts to document the cause precisely, then build the regression test to assert the ****honest measured served behavior**** and leave the PR open for owner review.

Let me first double-check one thing: whether the offline harness's "off" baseline (no merge, raw intervals) vs the production path's `merge_overlaps` could be confounding. The offline ``experiment.predict`` applies ``merge_overlaps(gap_tol=1.0)`` even in control. Let me verify my served runner's OFF path is the honest production OFF (no gate, full handoff). It is ? ``_select_for_handoff`` with ``min_crossings=0`` returns all windows, matching production ``wasb_hybrid``. Good.

Now let me write the regression test that asserts the honest served behavior, skipping when golden trajectories are absent.

49. user (2026-06-14T10:39:46.988Z)

```
1      """FusionSegmenter tests (mocked runner/provider/db ? no GPU/WSL).
2
3      Proves: net_crossings is persisted; the person cue is OFF by default (no person-feature
4      keys, PersonDetector never built); the persisted candidate set is the full superset of
the
5      substrate windows; served Gate A drops sub-threshold windows from the AI handoff while
6      persistence keeps the superset; and the person cue, when enabled, adds the person
features.
7
8      Also covers the PROMOTED Gate-A default (min_crossings=2): driven through a real
AppConfig
9      (the object main.py hands the segmenter), the held-shuttle window is dropped from the
handoff
10     by default, the real rally is kept, the persisted superset is unchanged, and
min_crossings=0
```

```

11     reverses it.
12     """
13     from types import SimpleNamespace
14     from unittest.mock import MagicMock, patch
15
16     from backend.config.models import AppConfig
17     from backend.pipeline.detectors.fusion_features import PERSON_FEATURE_NAMES
18     from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
19
20     # Window A [1,4]s: shuttle alternates across the net every frame -> many crossings (a
    rally).
21     # Window B [10,12]s: shuttle sits at the net level (held) -> 0 crossings. fps=30.
22     WIN_A = {"start": 1.0, "end": 4.0, "active_frame_count": 90, "peak_velocity": 0.4,
23             "mean_velocity": 0.2, "track_id_count": 2, "chunk_index": 0}
24     WIN_B = {"start": 10.0, "end": 12.0, "active_frame_count": 60, "peak_velocity": 0.1,
25             "mean_velocity": 0.05, "track_id_count": 1, "chunk_index": 0}
26
27
28     def _points():
29         pts = []
30         for f in range(30, 120):           # window A frames: y flips 100<->900 around the net
    (~500)
31             pts.append(SimpleNamespace(frame=f, visible=True, x=500.0, y=100.0 if f % 2 == 0
    else 900.0))
32         for f in range(300, 360):         # window B frames: y == net level -> deadband -> no
    crossings
33             pts.append(SimpleNamespace(frame=f, visible=True, x=500.0, y=500.0))
34         return pts
35
36
37     def _runner():
38         r = MagicMock()
39         r.healthcheck.return_value = (True, "env ok")
40         r.run_predict.return_value = "out.csv"
41         r.parse_trajectory_csv.return_value = _points()
42         r.trajectory_to_action_windows.return_value = [dict(WIN_A), dict(WIN_B)]
43         return r
44
45
46     def _run(indexing=None, provider_segments=None, config=None):
47         """Drive FusionSegmenter.process_video with a mocked runner/provider/db.
48
49         ``config`` (a real AppConfig or any object with ``.get``) takes precedence so a test
    can
50         exercise the SERVED path exactly as main.py builds it (config.get('indexing') -> the
    Pydantic-defaulted dict, where fusion.min_crossings defaults to 2). When omitted,
    the
51         bare ``{"indexing": indexing or {}}`` dict is used ? that path has NO fusion block,
    so the
52         in-code ``.get("min_crossings", 0)`` fallback applies (ungated), which is
    intentional for
53         the older mechanics tests below.
54         """
55

```

50. assistant (2026-06-14T10:39:51.456Z)

Now I understand the existing mock-test pattern. My regression test is complementary: it runs the **real served path on real cached trajectories** (the gap the task identified). Let me write it, mirroring the ablation litmus skip pattern. The test asserts the **honest measured served behavior** I just found ? not a fabricated "+0.074 holds" ? so it stays truthful and won't silently break if windowing changes.

51. assistant (2026-06-14T10:40:29.758Z)

"""SERVED Gate-A regression litmus ? the **production** FusionSegmenter path on real footage.

The flagged gap (#146): the served default flip (`default_segmenter=fusion_hybrid` +`min_crossings=2``) is **mock-tested only** on real video (`test_fusion_hybrid.py`` drives a MagicMock runner). The offline ablation litmus (`test_ablation.test_litmus_reproduces_gate_a``) validates the Gate-A *signal* ? but on the **permissive proposal windowing** (`distill_local.PROPOSAL``), NOT the production operating point. This module closes the gap: it runs the real `FusionSegmenter`` decision seams (`enrich_candidate_stats`` net-crossings + `select_for_handoff`` Gate A) over windows from the **production** `trajectory_to_action_windows`` config, on the 6 cached golden WASB trajectories, and asserts the served path's measured behaviour.

GPU-free / Gemini-free (person cue OFF) ? it stops at the AI-handoff boundary. Skips cleanly in CI where the golden trajectories are absent; runs on the owner box where they are present.

HONEST FINDING baked into the assertions (2026-06-14): on the **production windowing** Gate A is NOT the +0.074 win the offline harness reports. Production windowing is already conservative (velocity_threshold 0.02, merge_gap 2.0, min_window 2.0) ? few long windows, over-seg ratio already < 1 ? there is little held-shuttle junk left for Gate A to cut. So the served ?F1 is ~neutral (slightly negative), NOT +0.074. The +0.074 reproduces ONLY under the permissive proposal windowing (asserted in `test_proposal_windowing_reproduces_offline_lift`` below ? that is what the offline litmus measures). These tests pin BOTH facts so a future windowing change that silently revives ? or breaks ? either one trips the litmus.

"""

```
from __future__ import annotations
```

```
import json
import os
```

```
import pytest
```

cv2 is imported transitively by trajectory_hybrid (the served engine). On a box without (CI), skip the whole module rather than erroring at import.

```
pytest.importorskip("cv2")
```

```
from backend.eval import served_gate_a as sga # noqa: E402
from backend.pipeline.detectors.tracknet_runner import TrackNetRunner # noqa: E402
from backend.pipeline.detectors import rally_gate # noqa: E402
from backend.eval.calibrate_wasb import load_golden_gts # noqa: E402
from backend.eval.metrics import score # noqa: E402
from backend.eval import segmentation_metrics as _segm # noqa: E402
```

The golden manifest lives in the MAIN checkout's output/ (gitignored; absent in a fresh worktree / CI). Resolve it relative to this repo root first, with an env override for an out-of-tree checkout.

```
_REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
MANIFEST = os.environ.get(
    "SERVED_GATE_A_MANIFEST",
    os.path.join(_REPO_ROOT, "output", "human_lovo_manifest.json"),
)
```

```
def _golden_specs():
    """Return the manifest specs whose trajectory + label files are all present, else []."""
    if not os.path.isfile(MANIFEST):
        return []
    with open(MANIFEST, encoding="utf-8") as f:
        specs = json.load(f)
    ok = [s for s in specs
          if os.path.isfile(str(s.get("trajectory", "")))
          and os.path.isfile(str(s.get("labels", "")))]
    return ok if len(ok) >= 6 else []
```

```

_SPECS = _golden_specs()
_skip = pytest.mark.skipif(
    not _SPECS,
    reason=f"golden trajectories absent (manifest {MANIFEST!r}); runs on the owner box",
)

```

----- served-path litmus

```

@_skip
def test_served_path_runs_on_all_six_golden_videos():
    """The production FusionSegmenter decision path runs end-to-end (no GPU/Gemini) on all 6
    golden trajectories and yields a handoff window list for each, for both gate settings."""
    results = sga.evaluate_manifest(_SPECS)
    assert len(results) == 6
    for r in results:
        assert r.num_gt > 0
        assert r.n_handoff_off >= r.n_handoff_on # Gate A only ever drops windows, never adds

```

```

@_skip
def test_served_gate_a_does_not_regress_f1():
    """SERVED no-regression guard: on the production windowing Gate A must not MATERIALLY hurt
    F1. It is ~neutral here (the held-shuttle junk the offline harness sees is already merged
    away by the conservative production windowing) ? so we assert the mean ?F1 sits in a tight
    band around 0, never a real regression. (The +0.074 lift lives under the permissive proposal
    windowing ? see the dedicated test below.)"""
    results = sga.evaluate_manifest(_SPECS)
    agg = sga.aggregate(results)
    # Honest measured value (2026-06-14): mean served ?F1 ? -0.008. Pin a tolerance band so a
    # real regression (Gate A actively destroying served F1) trips, while the measured ~neutral
    # result passes.
    assert -0.03 <= agg["mean_delta_f1"] <= 0.03, agg
    # And Gate A must never collapse served F1 to ~0 on the production path.
    assert agg["mean_f1_on"] >= 0.5 * agg["mean_f1_off"] - 1e-9, agg

```

```

@_skip
def test_served_gate_a_does_not_increase_over_segmentation():
    """Gate A only ever DROPS windows, so the served over-seg ratio must not rise when it's on
    (the direction-of-effect invariant ? even where the absolute lift is ~neutral)."""
    results = sga.evaluate_manifest(_SPECS)
    agg = sga.aggregate(results)
    assert agg["mean_seg_ratio_on"] <= agg["mean_seg_ratio_off"] + 1e-9, agg
    # Per-video too: the gate is monotone (kept windows ? all windows).
    for r in results:
        assert r.seg_ratio_on <= r.seg_ratio_off + 1e-9, r

```

```

@_skip
def test_served_uses_production_windowing_operating_point():
    """Pin the served operating point to config.json's indexing defaults ? if someone retunes
    production windowing, this litmus's premise (and the divergence it documents) is revisited
    deliberately, not silently."""
    assert sga.PROD_VELOCITY_THRESH == 0.02
    assert sga.PROD_MERGE_GAP == 2.0
    assert sga.PROD_MIN_WINDOW == 2.0

```

----- the +0.074 lives under PROPOSAL windowing

distill_local.PROPOSAL ? the permissive, recall-oriented candidate windowing the OF

ablation/fusion_compare harness builds on (low velocity thresh, short merge/min-duration, small windows). The +0.074 Gate-A lift is a property of THIS windowing, not the served

```
_PROPOSAL = (0.005, 1.0, 0.5)
```

```
@_skip
```

```
def test_proposal_windowing_reproduces_offline_lift():
    """CONTROL / root-cause pin: under the OFFLINE harness's permissive proposal windowing, the
    SAME ``rally_gate`` net-crossing gate the served seam uses reproduces the offline litmus
    (?F1 ? +0.074, 6/6 wins, over-seg 1.68?1.01). This proves the served ~neutral result is a
    WINDOWING difference, not a broken gate ? and locks the offline number to its true
    premise."""
    d_on, d_off, seg_on, seg_off = [], [], [], []
    for s in _SPECS:
        fps, fw = float(s["fps"]), float(s["frame_width"])
        pts = TrackNetRunner.parse_trajectory_csv(s["trajectory"])
        wins = TrackNetRunner.trajectory_to_action_windows(
            pts, fps=fps, frame_width=fw, velocity_thresh=_PROPOSAL[0],
            merge_gap=_PROPOSAL[1], min_window_duration=_PROPOSAL[2])
        iv = [(w["start"], w["end"]) for w in wins]
        est = rally_gate.estimate_play_axis(pts)
        gated = rally_gate.gate_windows(pts, iv, fps, min_crossings=0, estimate=est)
        on = [(g.start, g.end) for g in gated if g.crossings >= 2]
        gts = load_golden_gts(s["labels"])
        d_on.append(score(on, gts).f1)
        d_off.append(score(iv, gts).f1)
        seg_on.append(_segm.segment_count_ratio(on, gts))
        seg_off.append(_segm.segment_count_ratio(iv, gts))
    n = len(d_on)
    mean_delta = (sum(d_on) - sum(d_off)) / n
    wins = sum(1 for a, b in zip(d_on, d_off) if a > b + 1e-9)
    assert mean_delta == pytest.approx(0.074, abs=0.01), mean_delta
    assert wins == 6, wins
    assert sum(seg_off) / n == pytest.approx(1.68, abs=0.05)
    assert sum(seg_on) / n == pytest.approx(1.01, abs=0.05)
```

52. user (2026-06-14T10:40:29.804Z)

File created successfully at: C:\Users\avidu\Projects\bhi-servedverify\tests\test_served_gate_a_regression.py (file state is current in your context ? no need to Read it back)

53. assistant (2026-06-14T10:40:33.749Z)

Now let me run the new test, then ruff + mypy + the relevant existing tests.

54. user (2026-06-14T10:40:41.157Z)

```
===== test session starts =====
platform win32 -- Python 3.13.13, pytest-9.0.3, pluggy-1.6.0 --
C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\avidu\Projects\bhi-servedverify
configfile: pytest.ini
plugins: anyio-4.13.0, cov-7.1.0
collecting ... collected 5 items

tests/test_served_gate_a_regression.py::test_served_path_runs_on_all_six_golden_videos SKIPPED [
20%]
tests/test_served_gate_a_regression.py::test_served_gate_a_does_not_regress_f1 SKIPPED [ 40%]
tests/test_served_gate_a_regression.py::test_served_gate_a_does_not_increase_over_segmentation
SKIPPED [ 60%]
tests/test_served_gate_a_regression.py::test_served_uses_production_windowing_operating_point
```

```
SKIPPED [ 80%]
tests/test_served_gate_a_regression.py::test_proposal_windowing_reproduces_offline_lift SKIPPED
[100%]
```

```
===== 5 skipped in 4.76s =====
```

55. assistant (2026-06-14T10:40:48.811Z)

The tests skipped ? the manifest in the worktree's `output/` is absent (gitignored), so my `\_REPO\_ROOT/output/human\_lovo\_manifest.json` resolution finds nothing. That's the correct CI-skip behavior. But to verify the tests actually PASS on the owner box (this machine), I need to point them at the main checkout's manifest via the `SERVED\_GATE\_A\_MANIFEST` env override I built in.

56. user (2026-06-14T10:41:07.419Z)

```
===== test session starts =====
platform win32 -- Python 3.13.13, pytest-9.0.3, pluggy-1.6.0 --
C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe
cachedir: .pytest_cache
rootdir: C:\Users\avidu\Projects\bhi-servedverify
configfile: pytest.ini
plugins: anyio-4.13.0, cov-7.1.0
collecting ... collected 5 items

tests/test_served_gate_a_regression.py::test_served_path_runs_on_all_six_golden_videos PASSED [ 20%]
tests/test_served_gate_a_regression.py::test_served_gate_a_does_not_regress_f1 PASSED [ 40%]
tests/test_served_gate_a_regression.py::test_served_gate_a_does_not_increase_over_segmentation PASSED [ 60%]
tests/test_served_gate_a_regression.py::test_served_uses_production_windowing_operating_point PASSED [ 80%]
tests/test_served_gate_a_regression.py::test_proposal_windowing_reproduces_offline_lift PASSED [100%]

===== 5 passed in 15.76s =====
```

57. assistant (2026-06-14T10:41:10.983Z)

All 5 tests pass on the owner box (with golden data) and skip cleanly without it. Now ruff + mypy on the two new files, and the existing fusion/ablation tests to confirm no regression.

58. user (2026-06-14T10:41:42.818Z)

```
=== RUFF ===
All checks passed!
=== MYPY ===
backend\sports\__init__.py:7: note: By default the bodies of untyped functions are not checked, consider using --check-untyped-defs [annotation-unchecked]
backend\providers\__init__.py:7: note: By default the bodies of untyped functions are not checked, consider using --check-untyped-defs [annotation-unchecked]
Success: no issues found in 1 source file
```

59. assistant (2026-06-14T10:41:48.529Z)

Ruff clean, mypy clean. Note one thing: my runner calls `seg.\_enrich\_candidate\_stats` / `seg.\_select\_for\_handoff` (protected members). Ruff passed, but let me check the project's mypy config covers strictness and whether accessing protected members from outside is flagged. mypy doesn't flag protected access by default and passed. Let me run the existing related tests plus the full suite to confirm no regression.

60. user (2026-06-14T10:42:14.081Z)

```
.....SS..... [100%]
```

43 passed, 2 skipped in 22.43s

61. assistant (2026-06-14T10:42:18.777Z)

All existing tests pass (2 skips are the golden-litmus tests that skip without golden JSONs in the worktree ? expected). Let me run the full offline test suite via the project's runner to be thorough, and also confirm mypy passes on the whole backend (in case my import touches typing elsewhere).

62. user (2026-06-14T10:43:14.534Z)

```
=====
Full test suite: C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe -m pytest
-q
obsreport (optional reporting dep): available
=====
.....SS..... [ 11%]
..... [ 23%]
..... [ 35%]
..... [ 46%]
..... [ 58%]
..... [ 70%]
.....SSSS..... [ 81%]
..... [ 93%]
..... [100%]
609 passed, 7 skipped in 53.05s
=====
RESULT: PASS - all tests green
=====
```

63. assistant (2026-06-14T10:43:18.844Z)

Full suite: 609 passed, 7 skipped (golden-data-gated, expected to skip in the worktree). Now let me run mypy across the eval package to be safe, then check ruff on the whole repo touch surface.

64. user (2026-06-14T10:43:23.934Z)

```
=== MYPY eval pkg ===
backend\sports\__init__.py:7: note: By default the bodies of untyped functions are not checked,
consider using --check-untyped-defs [annotation-unchecked]
backend\providers\__init__.py:7: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
backend\annotations\base.py:85: note: By default the bodies of untyped functions are not
checked, consider using --check-untyped-defs [annotation-unchecked]
Success: no issues found in 31 source files
=== RUFF repo ===
All checks passed!
```

65. assistant (2026-06-14T10:43:30.060Z)

Everything is green: ruff clean, mypy clean (31 source files), full suite 609 passed.

Now let me reconsider whether the test assertions are framed correctly given the honest finding. The task framing assumed Gate-A would HOLD (+0.074) on the served path. It does NOT ? there's a material divergence. My test correctly pins the ****measured truth**** (served ?F1 ~neutral, +0.074 only under proposal windowing). This is the honest engineering outcome. The verdict is ****DIVERGENCE****, not a clean HOLD, and per the task I must not merge a misleading "pass" ? I'll leave the PR open for owner review.

Let me capture the precise per-video candidate-count diagnostic to make the PR body and final report fully evidenced, then commit (branch-safety re-verify) and open the PR.

66. user (2026-06-14T10:43:36.161Z)

video	prod_wins	prop_wins
adarsh_avi_singles	97	172
kushagra_singles	2	20
largetest_doubles	52	86
gbaaddy	22	82
testlarge_short	5	14
mahadevpura_singles	24	57

67. assistant (2026-06-14T10:43:41.712Z)

This confirms the mechanism cleanly: production windowing produces far fewer candidate windows (e.g. gbaaddy 22 vs 82, kushagra 2 vs 20) because the conservative thresholds merge flight bursts and drop short ones. Fewer candidates ? less held-shuttle junk for Gate A to remove ? the offline lift doesn't transfer.

Now let me commit with branch-safety re-verification and open the PR. First re-verify HEAD and branch.

68. user (2026-06-14T10:43:43.661Z)

```
=== branch ===
feat/served-gate-a-regression
=== HEAD ===
e75ec35 feat(eval): LOS0/LOG0 ablation runner ? per-signal marginal ?F1 (Phase 0) (#147)
=== origin/master..HEAD (should be empty pre-commit) ===
=== status ===
?? backend/eval/served_gate_a.py
?? tests/test_served_gate_a_regression.py
```

69. assistant (2026-06-14T10:43:46.557Z)

Branch correct, HEAD is clean on origin/master, only my two new files untracked. Let me stage explicit paths and commit.

70. user (2026-06-14T10:43:47.416Z)

```
warning: in the working copy of 'backend/eval/served_gate_a.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'tests/test_served_gate_a_regression.py', LF will be replaced by
CRLF the next time Git touches it
A backend/eval/served_gate_a.py
A tests/test_served_gate_a_regression.py
```

71. user (2026-06-14T10:44:00.566Z)

```
[feat/served-gate-a-regression fd0d7b3] feat(eval): served Gate-A verification + repeatable
regression litmus (#146 follow-up)
 2 files changed, 359 insertions(+)
 create mode 100644 backend/eval/served_gate_a.py
 create mode 100644 tests/test_served_gate_a_regression.py
fd0d7b3 feat(eval): served Gate-A verification + repeatable regression litmus (#146 follow-up)
```

72. assistant (2026-06-14T10:44:04.198Z)

Only my commit is ahead of origin/master. Now push and open the PR. Per the task, since this is a DIVERGENCE (not a clean pass), I will ****leave the PR open for owner review**** rather than merging.

73. user (2026-06-14T10:44:07.384Z)

```
remote:
remote: Create a pull request for 'feat/served-gate-a-regression' on GitHub by visiting:
remote:      https://github.com/avidullu/badminton-highlight-indexer/pull/new/feat/served-gate-
a-regression
```



```
remote:
branch 'feat/served-gate-a-regression' set up to track 'origin/feat/served-gate-a-regression'.
To https://github.com/avidullu/badminton-highlight-indexer.git
* [new branch]      feat/served-gate-a-regression -> feat/served-gate-a-regression
```

74. user (2026-06-14T10:44:35.318Z)

<https://github.com/avidullu/badminton-highlight-indexer/pull/148>

75. assistant (2026-06-14T10:44:39.834Z)

PR #148 opened. Let me verify its state and confirm it's left open (not merged), and capture the PR number/URL for the report.

76. user (2026-06-14T10:44:42.207Z)

```
{"mergeable":"MERGEABLE","number":148,"state":"OPEN","title":"feat(eval): served Gate-A
verification + regression litmus ? DIVERGENCE finding (#146 follow-
up)","url":"https://github.com/avidullu/badminton-highlight-indexer/pull/148"}
```

77. assistant (2026-06-14T10:45:02.946Z)

PR #148 is OPEN and mergeable, left for owner review as required. The verification is complete.

Summary

I ran the **served Gate-A verification** on real footage and added a repeatable served-regression litmus. **Verdict: DIVERGENCE ? Gate A does NOT reproduce the offline +0.074 on the served (production) path.** I left the PR open for owner review rather than merging a misleading "pass."

What I did

- Set up isolated worktree `C:/Users/avidu/Projects/bhi-servedverify` off `origin/master` on branch `feat/served-gate-a-regression`.
- Drove the **real** `FusionSegmenter` decision seams (`\_enrich\_candidate\_stats` net-crossings + `\_select\_for\_handoff` Gate A) ? no reimplementing ? over the 6 cached golden WASB trajectories at the production windowing operating point. GPU-free, Gemini-free (person cue OFF).

Served-path numbers (production `FusionSegmenter`, IoU?0.5)

```
| video | gt | F1 off | F1 on | ?F1 | segR off ? on |
|---|---|---|---|---|---|
| adarsh_avi_singles | 96 | 0.632 | 0.628 | ?0.004 | 1.01 ? 0.79 |
| kushagra_singles | 32 | 0.000 | 0.000 | +0.000 | 0.06 ? 0.06 |
| largetest_doubles | 49 | 0.535 | 0.484 | ?0.051 | 1.06 ? 0.86 |
| gbaaddy | 48 | 0.086 | 0.088 | +0.003 | 0.46 ? 0.42 |
| testlarge_short | 6 | 0.182 | 0.182 | +0.000 | 0.83 ? 0.83 |
| mahadevpura_singles | 31 | 0.364 | 0.370 | +0.007 | 0.77 ? 0.74 |
| MEAN | | 0.300 | 0.292 | 0.008 | 0.70 ? 0.62 . 2/6 wins |
```

Did it match the offline +0.074? No.

- **Served / production windowing** (`vt=0.02, gap=2.0, min=2.0`): **?F1 ?0.008, 2/6 wins, over-seg already 0.70 (under one pred per rally)** ? Gate A is ~neutral, slightly negative.
- **Offline proposal windowing** (`vt=0.005, gap=1.0, min=0.5`): **+0.0739, 6/6, p=0.031, over-seg 1.68?1.01** ? reproduced exactly (the existing ablation litmus also reproduces it from the persisted JSONs).

Root cause (isolated, not a bug)

The +0.074 is a property of the **offline harness's permissive windowing**, not Gate A in isolation. I proved this: applying the **same** `rally\_gate` net-crossing gate to proposal-windowed candidates reproduces +0.074 exactly. Production windowing is far more conservative ? it merges flight bursts into few long windows and drops short ones, yielding **far fewer candidates** (gbaaddy 22 vs 82; kushagra 2 vs 20). Over-seg is already < 1, so there's little

held-shuttle junk left for Gate A to cut, and it occasionally trims a borderline true window. The flip is reversible (``min_crossings: 0` / `default_segmenter: wasb_hybrid``).

Tests added

- ``backend/eval/served_gate_a.py`` ? runner helper (production handoff selection on a cached trajectory; CLI + library API).
- ``tests/test_served_gate_a_regression.py`` ? 5 tests that **skip** cleanly without golden trajectories** (CI) and **run** on the owner box**. Assert the served no-regression band + monotone over-seg invariants, pin the production operating point, and a control test that reproduces the +0.074 under proposal windowing. All assert the **honest measured** behaviour.

Verify (all green)

- ``ruff check`` clean; ``mypy backend/eval/`` clean (31 files).
- Full suite: **609** passed, **7** skipped**. New litmus: 5 passed on owner box, 5 skip without golden data.

Merge status

PR #148 ? OPEN, left for owner review** (<https://github.com/avidullu/badminton-highlight-indexer/pull/148>). Branch-safe: ``git log origin/master..HEAD`` = only my one commit. The PR body lays out the three owner options (keep the ~neutral flip / align served windowing toward the proposal point so the lift transfers / revert the default).

Note: the new test resolves the golden manifest from the worktree's gitignored ``output/`` (absent ? skip); on the owner box it's run via ``SERVED_GATE_A_MANIFEST`` pointing at the main checkout. If the owner wants it to auto-run on this machine without the env var, the manifest would need to live at a tracked/known path.